

“Recipe Finder”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

MONISHA RAGHINI CHELLE :2520030537

SHAIK SABIYA TABASSUM :2520030502

K. SIRI CHANDANA :2520030453

Under the Guidance of

Dr. V. Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled “**Recipe Finder**” is a record of Bonafide work of **MONISHA RAGHINI CHELLE – 2520030537, SHAIK SABIYA TABASSUM – 2520030502, K.SIRI CHANDANA – 2520030453** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

MONISHA RAGHINI CHELLE – 2520030537

SHAIK SABIYA TABASSUM – 2520030502

K.SIRI CHANDANA – 2520030453

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled “**Recipe Finder**” is a bonafide work done and submitted by **MONISHA RAGHINI CHELLE – 2520030537, SHAIK SABIYA TABASSUM – 2520030502, K.SIRI CHANDANA – 2520030453** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27**.

Signature of the Guide
(Dr. V.Sireesha)

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, **Dr. V. Sireesha, Professor, Department of Computer Science and Engineering**, for her valuable guidance, suggestions, and continuous support throughout the development of the Recipe Finder project.

We also express our sincere thanks to **Dr. P. Venkateswara Rao, Associate Professor and HOD, Department of Computer Science and Engineering**, for providing the necessary academic support and facilities for completing this project.

We are grateful to the Department of Computer Science and Engineering for providing an environment that enabled us to apply Data Structures and Algorithms to a practical recipe-search problem.

Finally, we thank all the team members for their cooperation and contribution in algorithm implementation, dataset preparation, Java Swing interface development, testing, documentation, and presentation.

MONISHA RAGHINI CHELLE – 2520030537

SHAIK SABIYA TABASSUM – 2520030502

K.SIRI CHANDANA – 2520030453

Abstract

The **Recipe Finder** is a Java-based Data Structures and Algorithms application developed to search and retrieve recipes from a large text-based recipe corpus.

The current dataset contains **100,000 recipe records** stored in data/recipes.txt. Each recipe record contains a recipe name, ingredients, instructions, preparation time, and cooking time.

The system integrates multiple algorithmic techniques including **Naive Pattern Matching, Knuth-Morris-Pratt (KMP), Rabin-Karp, Aho-Corasick, Trie-based Autocomplete, Levenshtein Edit Distance, Bipartite Matching, Greedy Set Cover, Randomized Recommendation, and Parallel Similarity Processing.**

Trie-based autocomplete provides prefix suggestions while the user types. Exact recipe searching uses case-insensitive matching and verifies matching using the implemented pattern-matching algorithms. When an exact match is not found, fuzzy searching uses Levenshtein similarity. Ingredient-based searching uses bipartite matching, while greedy set cover is used to cover requested ingredients through selected recipes.

A **Java Swing graphical user interface** provides a normal search-oriented interface for the application. The algorithms are used internally and are not exposed through an algorithm-selection menu.

The project demonstrates the practical application of Data Structures and Algorithms to searching, matching, similarity calculation, optimization, and recommendation on a large recipe dataset.

Table Of Contents

Abstract.....	5
CHAPTER 1 - INTRODUCTION	11
1.1 Overview	11
1.2 Problem Statement.....	13
1.3 Scope of the Project.....	15
1.4 Objectives	16
CHAPTER 2 - SYSTEM REQUIREMENTS	18
2.1 Functional Requirements.....	18
2.1.1 Recipe Dataset Loading.....	18
2.1.2 Recipe Record Parsing.....	18
2.1.3 Recipe Storage	18
2.1.4 Trie-Based Autocomplete.....	18
2.1.5 Prefix Suggestions	19
2.1.6 Case-Insensitive Recipe Search.....	19
2.1.7 Naive Pattern Matching.....	19
2.1.8 KMP Search.....	19
2.1.9 Rabin-Karp Search	19
2.1.10 Aho-Corasick Pattern Verification	19
2.1.11 Levenshtein Distance Calculation	19
2.1.12 Fuzzy Search.....	20
2.1.13 Similarity Percentage.....	20
2.1.14 Ingredient Input	20
2.1.15 Bipartite Ingredient Matching	20
2.1.16 Greedy Ingredient Coverage.....	20
2.1.17 Random Recipe Recommendation	21
2.1.18 Parallel Similarity Processing.....	21
2.1.19 Complete Recipe Information	21
2.1.20 Input Validation.....	21
2.1.21 Java Swing Graphical Interface	21
2.2 Non-Functional Requirements.....	22
2.2.1 Usability.....	22

2.2.2 Performance.....	22
2.2.3 Scalability	22
2.2.4 Reliability	23
2.2.5 Maintainability.....	23
2.2.6 Portability	24
2.2.7 Data Consistency	24
2.2.8 Error Handling.....	24
2.2.9 Extensibility.....	24
CHAPTER 3 - SYSTEM DESIGN	25
3.1 High-Level Architecture.....	25
3.2 Data and Component Description.....	27
3.2.1RecipeFinder.java	27
3.2.2RecipeFinderGUI.java	27
3.2.3 Recipe Dataset	28
3.3 Search and Autocomplete Design.....	28
3.3.1 Trie-Based Autocomplete.....	28
3.3.2 Exact Search	29
3.3.3 Pattern Matching.....	29
3.3.4 Fuzzy Search.....	29
3.4 DSA and CO Mapping	30
3.5 Dataset Design.....	30
CHAPTER 4 - IMPLEMENTATION	32
4.1 Module-Wise Implementation.....	32
4.2 PSEUDOCODE.....	36
4.3 JAVA SWING INTERFACE.....	42
CHAPTER 5 - RESULTS	47
5.1 Testing Strategy.....	47
5.2 Functional Results	47
5.2.1Exact Search	47
5.2.2Case-Insensitive.....	49
5.2.3Autocomplete.....	50
5.2.4Fuzzy Search.....	51
5.2.5Ingredient Search.....	52

5.2.6 Greedy Coverage	53
5.2.7 Random.....	54
5.2.8 Aho-Corasick Pattern Matching	55
5.2.9 Empty Input Validation	56
5.2.10 Exit Operation.....	57
5.3 GUI Results	58
CHAPTER 6 — CHALLENGES & LIMITATIONS	64
6.1 Issues Faced During Development	64
6.1.1 Integrating Multiple Pattern-Matching Algorithms.....	64
6.1.2 Handling a Large Dataset	64
6.1.3 Autocomplete.....	65
6.1.4 Dataset Parsing	65
6.1.5 Fuzzy Search.....	66
6.1.6 Ingredient Matching	66
6.1.7 Greedy Coverage	67
6.1.8 GUI Integration.....	67
6.2 Solutions Applied	68
6.2.1 Solution for Multiple Pattern-Matching Algorithms	68
6.2.2 Solution for Handling the 100,000-Record Dataset	68
6.2.3 Solution for Autocomplete	69
6.2.4 Solution for Dataset Parsing	69
6.2.5 Solution for Fuzzy Search	69
6.2.6 Solution for Ingredient Matching	70
6.2.7 Solution for Ingredient Coverage	70
6.2.8 Solution for Random Recommendation	71
6.2.9 Solution for Parallel Similarity Processing	71
6.2.10 Solution for Java Swing Integration	72
6.2.11 Solution for Invalid and Empty Input.....	73
6.2.12 Overall Effect of the Solutions	73
6.3 Limitations.....	74
6.3.1 Text-File-Based Data Storage	74
6.3.2 Aho-Corasick Implementation	75
6.3.3 Parallel Fuzzy-Search Overhead	75

6.3.4 Greedy Set Cover Approximation	75
6.3.5 Desktop Application Platform	75
6.3.6 Dataset Provenance.....	75
6.3.7 Summary of Limitations.....	76
CHAPTER 7 — CONCLUSION & FUTURE WORK	77
7.1 Conclusion.....	77
7.2 FUTURE WORK	78
7.2.1 Improved Search Ranking	78
7.2.2 Personalized Recipe Recommendations	78
7.2.3 Nutritional and Dietary Information.....	78
7.2.4 Larger Real-World Datasets	78
7.2.5 Graph-Based Recipe and Ingredient Relationships	79
7.2.6 Web-Based or Mobile Interface	79
7.2.7 Automated Performance Benchmarking	79
7.2.8 Larger Automated Test Suites	79
7.2.9 Optimization of Parallel Similarity Processing	79
7.3 SKILLS LEARNED	79
7.3.1 Problem Analysis.....	79
7.3.2 Algorithm Selection.....	79
7.3.3 String Matching	80
7.3.4 Trie Implementation	80
7.3.5 Dynamic Programming.....	80
7.3.6 Edit-Distance Calculation.....	80
7.3.7 Bipartite Matching.....	80
7.3.8 Greedy Optimization	80
7.3.9 Randomized Algorithms.....	80
7.3.10 Java Concurrency	80
7.3.11 Java Swing.....	80
7.3.12 Dataset Parsing	81
7.3.13 Testing	81
7.3.14 Git and GitHub	81
7.3.15 Technical Documentation.....	81
REFERENCES	82

List of Figures

Fig. 1.1 Overview of the Recipe Finder System

Fig. 3.1 High-Level Architecture of the Recipe Finder System

Fig. 3.2 System Flowchart

Fig. 3.3 DSA and CO Mapping

Fig. 3.4 Recipe Dataset Record Structure

Fig. 5.1 Exact Recipe Search Execution

Fig. 5.2 Case-Insensitive Recipe Search

Fig. 5.3 Trie-Based Autocomplete

Fig. 5.4 Fuzzy Search Using Levenshtein Similarity

Fig. 5.5 CO4 Ingredient Matching Using Bipartite Matching

Fig. 5.6 CO5 Greedy Set Cover Execution

Fig. 5.7 CO6 Random Recipe Recommendation

Fig. 5.8 Aho-Corasick Pattern Matching Execution

Fig. 5.9 Empty Input Validation

Fig. 5.10 Application Exit Operation

Fig. 5.11 Java Swing Recipe Finder Dashboard

Fig. 5.12 Recipe Search and Autocomplete

Fig. 5.13 CO4 Ingredient Search

Fig. 5.14 CO5 Greedy Coverage

Fig. 5.15 CO6 Random Recommendation

Fig. 6.1 Solutions Applied in Recipe Finder Development

CHAPTER 1 - INTRODUCTION

1.1 Overview

Recipe Finder is a Data Structures and Algorithms project that applies different algorithmic techniques to the problem of searching, retrieving, matching, and recommending recipes from a large recipe corpus. The project combines string matching, prefix searching, dynamic programming, graph-based matching, greedy optimization, randomization, and parallel processing within a single application.

The application works with a text-based recipe corpus stored in:

data/recipes.txt

The current dataset contains 100,000 recipe records. Each recipe is processed and converted into a structured representation by RecipeFinder.java. The parsed information is organized into Recipe objects so that the searching and recommendation operations can work with structured recipe information instead of repeatedly processing the raw text file.

Each Recipe object stores important information such as:

recipe name;

complete recipe text;

normalized ingredients.

The dataset-loading and parsing stage forms the initial part of the system. After the recipe records are loaded, the application prepares the required data structures and search mechanisms. This allows the different operations to work on the loaded collection without repeatedly reading the complete dataset for every query.

One of the important features of the system is prefix-based autocomplete. The application builds a Trie over recipe names. A Trie is suitable for prefix searching because recipe names can be organized according to their individual characters. When a user enters a partial recipe name, the system can traverse the corresponding prefix and generate matching recipe-name suggestions.

For complete recipe queries, the application performs exact recipe-name checking and verifies the pattern using several pattern-matching techniques. The implemented techniques include:

Naive Pattern Matching;

Knuth-Morris-Pratt (KMP);

Rabin-Karp;

Aho-Corasick.

The inclusion of multiple pattern-matching algorithms allows the project to demonstrate

different approaches to string searching. Naive Pattern Matching performs direct comparisons, KMP uses prefix information to avoid unnecessary comparisons, Rabin-Karp uses hashing for pattern comparison, and Aho-Corasick uses a Trie-based matching structure with failure relationships.

The application also supports searches where the user's input may not exactly match a stored recipe name. When an exact recipe is not found, the system uses Levenshtein-based similarity and parallel fuzzy-search processing. Levenshtein Edit Distance determines the number of insertions, deletions, and substitutions required to transform one string into another. The calculated distance can then be used to determine how closely the query resembles a recipe name.

In addition to recipe-name searching, the project provides ingredient-based operations. These operations address situations where the user is interested in recipes based on available ingredients rather than knowing the exact recipe name. CO4 uses bipartite matching to compare requested ingredients with recipe ingredients. This represents the ingredient comparison as a matching problem between two sets.

CO5 uses a greedy set-cover strategy for ingredient coverage. The requested ingredients are treated as elements that need to be covered, and recipes are selected according to the useful coverage they provide. The process continues by updating the set of currently uncovered ingredients until the required coverage is achieved or no further useful selection is available.

CO6 provides randomized recommendation and parallel similarity processing. The randomized recommendation feature allows the system to select a recipe without requiring a specific recipe-name query. Parallel similarity processing is used for independent similarity calculations through Java concurrency mechanisms.

The user-facing part of the application is provided through a Java Swing graphical user interface. The GUI allows users to interact with the implemented DSA functionality without directly interacting with the individual algorithm implementations. Search, autocomplete, ingredient-based operations, greedy coverage, and random recommendation can therefore be accessed through a common application interface.

The overall system demonstrates how several individual Data Structures and Algorithms can be integrated into a practical application. Rather than using one algorithm for every operation, the project uses different techniques according to the nature of each problem. This provides a practical connection between the theoretical concepts studied in Data Structures and Algorithms and their implementation in a Java-based application.

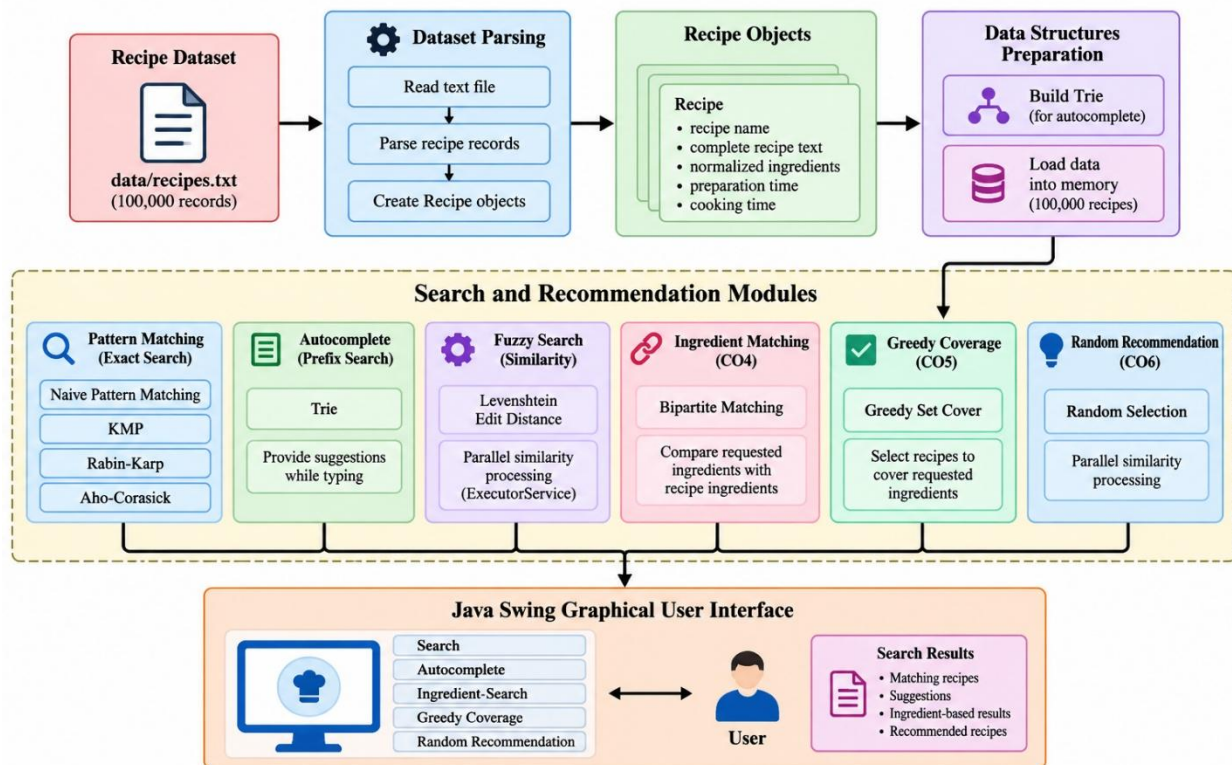


Fig. 1.1 Overview of the Recipe Finder System

Fig. 1.1 Overview of the Recipe Finder System

The system begins with the recipe dataset, followed by parsing and data preparation. The prepared data is then used by the Trie, pattern-matching, fuzzy-search, ingredient-matching, coverage, and recommendation components. The results of these operations are presented to the user through the Java Swing interface.

1.2 Problem Statement

Searching a large recipe corpus using only exact string comparison can be restrictive when users enter partial recipe names, different capitalization, or slightly misspelled recipe names. A user may not always remember the complete name of a recipe and may instead enter only a portion of the name.

For example, a user may enter a partial query such as:

cak

instead of entering the complete recipe name. In such cases, a simple exact comparison may not provide the desired result. A prefix-search mechanism can instead identify recipe names beginning with the entered characters and provide suitable suggestions.

Similarly, users may enter the same recipe name using different capitalization. Therefore, case-insensitive processing is required to make the search operation more flexible.

Another challenge occurs when the entered recipe name contains a spelling variation. Exact matching cannot directly identify a recipe if the input differs from the stored name. Fuzzy searching addresses this problem by calculating the similarity between the query and candidate recipe names.

Searching by recipe name is also not sufficient for every user requirement. Users may have ingredients available at home and want to identify recipes that can be prepared using those ingredients. This creates a different computational problem because multiple ingredients must be compared with the ingredients associated with recipes.

Ingredient coverage introduces another requirement. A user may provide several ingredients and want a set of recipes that collectively covers those ingredients. This can be treated as a set-cover-style problem and addressed using a greedy selection strategy.

Therefore, the project addresses the problem of developing a recipe-search application that combines multiple Data Structures and Algorithms to support different types of recipe retrieval and recommendation operations while operating on a **100,000-record recipe dataset**.

The major requirements addressed by the project include:

- exact recipe searching;
- case-insensitive searching;
- prefix-based autocomplete;
- pattern matching;
- fuzzy searching;
- similarity calculation;
- ingredient matching;
- ingredient coverage;
- random recommendation;
- parallel similarity processing.

The project also aims to provide a graphical interface through Java Swing so that the implemented algorithms can be accessed through a simple user-oriented application.

The problem is therefore not limited to finding an exact recipe name. It involves supporting several forms of search and matching while maintaining a structured implementation that demonstrates the use of different DSA techniques.

1.3 Scope of the Project

The scope of the Recipe Finder project covers the development of a Java-based recipe-search and recommendation application using multiple Data Structures and Algorithms.

The project includes recipe dataset loading and parsing, where the records stored in data/recipes.txt are read and converted into Recipe objects. The current dataset contains 100,000 records and is used as the primary dataset for testing the application's functionality.

The project covers recipe-name searching, including exact and case-insensitive searches. These operations allow users to retrieve recipes based on their names without requiring them to manually inspect the complete dataset.

The project also includes Trie-based autocomplete. Recipe names are inserted into a Trie, allowing prefix-based suggestions to be generated when the user enters partial input.

The scope also includes four pattern-matching techniques:

Naive Pattern Matching

Knuth-Morris-Pratt (KMP)

Rabin-Karp

Aho-Corasick

These algorithms provide the pattern-matching component of the application and allow the project to demonstrate multiple approaches to string processing.

For approximate matching, the scope includes Levenshtein Edit Distance, fuzzy searching, and similarity calculation. These features help process queries that do not exactly match the stored recipe names.

The project also covers ingredient-based operations. Bipartite matching is applied to ingredient comparison, while Greedy Set Cover is applied to ingredient coverage. These operations demonstrate the application of graph-based matching and greedy optimization techniques to recipe-related problems.

The scope further includes random recipe recommendation. The system can select a recipe randomly from the loaded collection when the user requests a recommendation without providing a specific recipe name.

Another component within the scope is parallel fuzzy similarity processing. Independent similarity calculations can be processed using Java concurrency mechanisms, providing practical exposure to parallel processing.

The project also includes a Java Swing graphical interface. The GUI acts as the interaction layer through which users can access recipe searching, autocomplete, ingredient operations, greedy coverage, and random recommendation.

Testing is also included within the scope of the project. The implemented functionality is tested using different inputs, including exact recipe names, different capitalization, partial names, fuzzy queries, ingredient combinations, empty input, invalid input, and recommendation requests.

The scope therefore covers the complete flow from dataset loading and algorithmic processing to user interaction and result presentation.

1.4 Objectives

The main objectives of the Recipe Finder project are as follows:

1. **To develop a Java-based recipe-search application** capable of processing a large recipe dataset and providing recipe retrieval functionality.
2. **To implement multiple pattern-matching algorithms**, including Naive Pattern Matching, KMP, Rabin-Karp, and Aho-Corasick, and integrate them into the recipe-search system.
3. **To provide Trie-based autocomplete** so that recipe suggestions can be generated from partial recipe-name input.
4. **To implement fuzzy searching using Levenshtein Edit Distance** to support recipe-name queries that are not exact matches.
5. **To calculate similarity between a query and recipe names** so that approximate matches can be identified.
6. **To implement ingredient-based recipe matching** so that users can search for recipes using available ingredients.
7. **To apply bipartite matching for ingredient comparison** and demonstrate the use of graph-based matching in the recipe domain.
8. **To implement Greedy Set Cover for ingredient coverage**, allowing the system to select recipes according to their ability to cover requested ingredients.
9. **To provide randomized recipe recommendations** for users who want a recipe suggestion without entering a specific recipe name.
10. **To implement parallel similarity processing** using Java concurrency mechanisms for independent similarity calculations.
11. **To process a 100,000-record recipe dataset** and provide a larger environment for testing the implemented algorithms and data structures.
12. **To provide a Java Swing graphical interface** that allows users to interact with the recipe-search and recommendation functionality.
13. **To analyze the complexity of the implemented algorithms** and understand their

computational requirements.

14. **To integrate different DSA techniques into one application**, demonstrating how different algorithms can be selected according to different computational requirements.
15. **To test the major functionalities of the application** using representative inputs and verify that the implemented operations produce the expected results.

CHAPTER 2 - SYSTEM REQUIREMENTS

2.1 Functional Requirements

Functional requirements describe the operations and services that the Recipe Finder system is expected to provide to users. The system combines dataset processing, searching, autocomplete, fuzzy matching, ingredient-based operations, recommendation, and graphical interaction. The following functional requirements define the expected behaviour of the application.

2.1.1 Recipe Dataset Loading

The system shall load recipe records from:

data/recipes.txt

The dataset currently contains 100,000 recipe records. The application shall read the dataset during initialization and make the loaded recipe information available to the different search and recommendation operations.

Loading the dataset at initialization avoids repeatedly reading the complete text file for every user query. This also provides a common collection of recipe records for the different algorithms implemented in the system.

2.1.2 Recipe Record Parsing

The system shall parse individual recipe records from the dataset. Each record contains structured recipe information such as the recipe name, ingredients, instructions, preparation time, and cooking time.

The parser shall identify the boundaries of each recipe and associate the corresponding fields with the correct recipe object. The parsed information shall then be used by the search and recommendation components.

2.1.3 Recipe Storage

The system shall store the parsed recipes using Java data structures. The stored collection shall allow the application to access recipe names and complete recipe information during subsequent operations.

The stored recipe information is used by exact searching, fuzzy searching, ingredient matching, greedy coverage, and random recommendation.

2.1.4 Trie-Based Autocomplete

The system shall build a Trie using recipe names to support prefix-based searching.

The Trie shall organize recipe names according to their character prefixes. Once the Trie has been constructed, the application shall use the entered prefix to locate corresponding recipe suggestions.

The Trie shall be constructed once during initialization rather than rebuilt for every character entered by the user.

2.1.5 Prefix Suggestions

The system shall display recipe-name suggestions based on the prefix entered by the user.

For example, when a user enters a partial recipe name, the application shall identify recipe names beginning with that prefix and display the available suggestions. This allows the user to select or complete a recipe name without entering the entire name manually.

2.1.6 Case-Insensitive Recipe Search

The system shall support case-insensitive recipe-name searching.

For example, queries entered using different capitalization should still be compared appropriately with the stored recipe names. This prevents capitalization differences from unnecessarily preventing a matching recipe from being identified.

2.1.7 Naive Pattern Matching

The system shall implement Naive Pattern Matching as one of the pattern-searching techniques.

The algorithm shall compare the required pattern against the available text through direct character comparisons. The implementation provides a straightforward pattern-matching method and forms part of the project's demonstration of different string-searching techniques.

2.1.8 KMP Search

The system shall implement the Knuth-Morris-Pratt (KMP) pattern-matching algorithm.

The implementation shall use the Longest Prefix Suffix (LPS) concept to reduce unnecessary character comparisons during pattern matching. The KMP implementation forms part of the pattern-matching functionality of the Recipe Finder system.

2.1.9 Rabin-Karp Search

The system shall implement the Rabin-Karp pattern-matching algorithm.

The algorithm shall use hashing to compare the pattern with portions of the input text. A rolling-hash mechanism shall be used as part of the implementation.

2.1.10 Aho-Corasick Pattern Verification

The system shall provide an implementation of Aho-Corasick pattern verification.

The implementation shall use a Trie-like structure together with failure relationships to process the supplied pattern. This functionality provides an additional pattern-matching technique within the Recipe Finder system.

2.1.11 Levenshtein Distance Calculation

The system shall calculate Levenshtein Edit Distance between strings.

The calculation shall consider character insertion, deletion, and substitution operations. The resulting distance shall be used by the fuzzy-search component to determine the closeness between a user query and a recipe name.

2.1.12 Fuzzy Search

The system shall provide fuzzy recipe searching when an exact recipe-name match is not obtained.

The fuzzy-search functionality shall compare the entered query with candidate recipe names using Levenshtein-based similarity. This allows the system to process approximate or slightly different recipe-name queries.

2.1.13 Similarity Percentage

The system shall calculate a similarity percentage between a query and a recipe name.

The similarity calculation is based on the Levenshtein distance and the maximum string length. The project uses the following calculation:

Similarity =

$$(1 - \text{Edit Distance} / \text{Maximum String Length}) \times 100$$

The calculated value can be used to determine how closely a recipe name matches the user's query.

2.1.14 Ingredient Input

The system shall accept comma-separated ingredients as user input.

For example:

chicken,onion,tomato

The entered ingredients shall be processed and compared with the ingredient information stored in the recipe records.

2.1.15 Bipartite Ingredient Matching

The system shall perform ingredient matching using a bipartite matching approach.

The requested ingredients and recipe ingredients can be treated as two sets, with matching relationships established between compatible ingredients. The result shall provide information about the matched ingredients.

This functionality represents the ingredient-matching operation associated with CO4.

2.1.16 Greedy Ingredient Coverage

The system shall perform ingredient coverage using a Greedy Set Cover strategy.

The requested ingredients shall initially be treated as uncovered elements. The system shall iteratively select recipes that provide useful coverage of the currently uncovered ingredients and update the remaining uncovered set.

This functionality represents the ingredient-coverage operation associated with CO5.

2.1.17 Random Recipe Recommendation

The system shall provide a randomized recipe recommendation feature.

When the user requests a random recommendation, the application shall select a recipe from the loaded recipe collection and display the selected recipe.

The operation does not require the user to provide a specific recipe name.

2.1.18 Parallel Similarity Processing

The system shall support parallel processing of independent similarity calculations.

Java concurrency mechanisms such as `ExecutorService` and `Future` shall be used to submit and collect independent similarity-processing tasks.

This functionality provides practical implementation of parallel processing within the recipe-search system.

2.1.19 Complete Recipe Information

The system shall display the available information for a selected recipe.

The displayed information may include:

- recipe name;
- ingredients;
- instructions;
- preparation time;
- cooking time.

This allows the user to view the recipe after a successful search or recommendation operation.

2.1.20 Input Validation

The system shall handle invalid and empty input appropriately.

If the user submits an empty recipe query, the system shall display an appropriate message instead of attempting an unnecessary search. Similarly, invalid recipe names and invalid ingredient input shall be handled without causing unexpected termination of the application.

2.1.21 Java Swing Graphical Interface

The system shall provide a Java Swing graphical user interface.

The GUI shall provide access to the major user-oriented functions of the Recipe Finder application, including recipe search, autocomplete, ingredient search, greedy coverage, and random recommendation.

The GUI shall act as the interaction layer while the core DSA operations remain implemented in the Recipe Finder program.

2.2 Non-Functional Requirements

Non-functional requirements describe the quality characteristics and operational expectations of the Recipe Finder system. These requirements focus on usability, performance, scalability, reliability, maintainability, and portability.

2.2.1 Usability

The application should provide a simple search-oriented interface that can be used without requiring the user to understand the internal algorithms.

The Java Swing interface should allow users to enter recipe names or ingredients and access the available operations through clearly organized controls.

Autocomplete suggestions should assist users while entering recipe names. Search results should present recipe information in a readable form.

The system should also provide appropriate messages for empty or invalid input so that users understand what action is required.

2.2.2 Performance

The application should avoid unnecessary repeated processing of the dataset.

The **100,000-record dataset should be loaded once**, after which the parsed recipes can be reused by different operations. Similarly, the Trie should be built once rather than reconstructed for every keystroke.

This approach reduces repeated initialization work and allows the application to respond to subsequent user operations using the already prepared data structures.

The project also includes parallel similarity processing for independent similarity calculations. However, the practical performance benefit of parallel processing can depend on workload size and thread-management overhead.

2.2.3 Scalability

The application should support the current **100,000-record dataset** and maintain the same overall processing structure as the dataset grows.

The system uses reusable in-memory data structures and constructs the Trie during initialization. These design choices provide a basis for handling a larger recipe collection without changing the overall application workflow.

Future expansion may require additional optimization if the dataset becomes significantly larger.

2.2.4 Reliability

The system should handle common input and execution problems without unexpected termination.

The application should appropriately handle:

- empty input;
- invalid recipe names;
- missing dataset files;
- invalid ingredient input.

If a requested recipe cannot be found, the system should provide an appropriate result rather than terminating. Similarly, invalid or incomplete user input should be handled through validation.

Dataset loading and parsing should also be performed carefully so that recipe information is associated with the correct records.

2.2.5 Maintainability

The project should be organized so that the core DSA implementation and graphical interface can be maintained separately.

The main DSA implementation is maintained in:

src/RecipeFinder.java

This file contains the core recipe-processing and algorithmic functionality.

The graphical interface is implemented separately in:

src/RecipeFinderGUI.java

Separating the GUI from the core DSA implementation makes it easier to modify the user interface without unnecessarily changing the underlying algorithms.

The modular organization also allows individual algorithm implementations to be tested and maintained independently.

Clear naming, structured methods, comments where necessary, and separation of responsibilities further support maintainability.

2.2.6 Portability

The project uses standard **Java and Java Swing**, which allows the application to run on systems that have a compatible Java Development Kit (JDK) installed.

The application does not depend on a specialized web framework or platform-specific GUI technology. The dataset is maintained as a text file within the project structure, allowing the application to access the required recipe information from the project directory.

The use of standard Java libraries also makes the project easier to compile, execute, and transfer between compatible development environments.

2.2.7 Data Consistency

The recipe dataset should maintain a consistent record structure so that the parser can correctly identify recipe names, ingredients, instructions, preparation time, and cooking time.

Consistency is particularly important because the current dataset contains 100,000 records. An inconsistent record format can affect parsing and consequently influence search and recommendation operations.

Therefore, the dataset structure should remain consistent when records are added or modified.

2.2.8 Error Handling

The system should provide controlled responses when expected input or resources are unavailable.

For example, if the recipe dataset cannot be located, the application should report the problem rather than silently continuing with an incomplete dataset. Similarly, invalid user queries should result in an informative response.

Error handling improves the overall reliability of the application and helps users understand problems during execution.

2.2.9 Extensibility

The system should allow additional algorithms and functionality to be incorporated in future versions.

Because the core DSA implementation and GUI are maintained separately, additional search or recommendation features can be introduced without completely redesigning the application.

This structure provides a foundation for possible future extensions such as improved result ranking, additional recommendation techniques, larger datasets, automated benchmarking, and alternative user interfaces.

CHAPTER 3 - SYSTEM DESIGN

3.1 High-Level Architecture

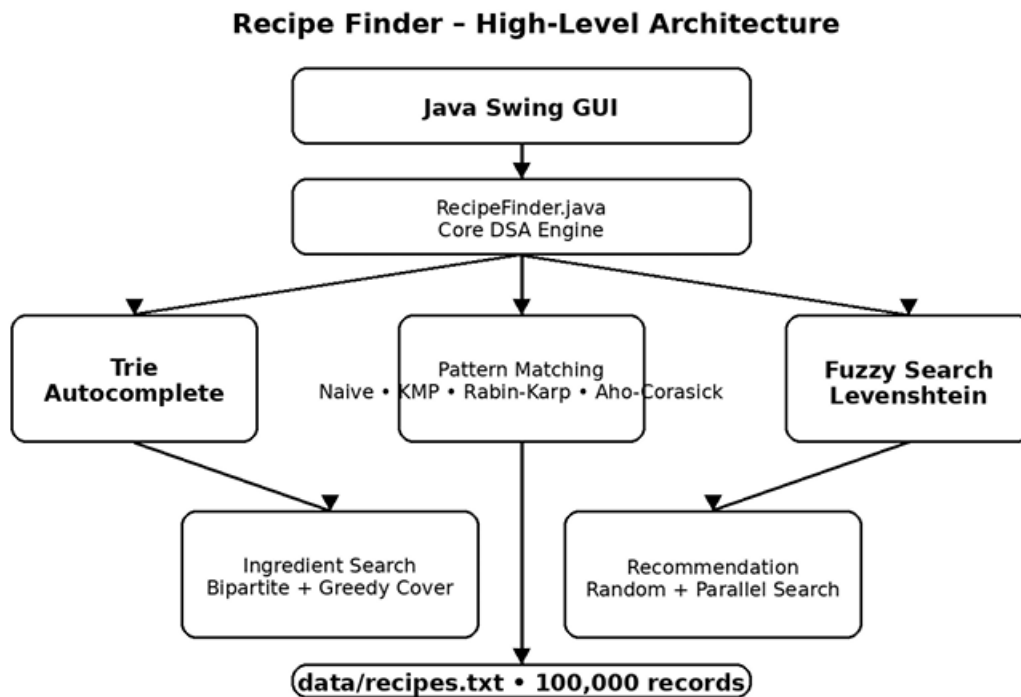


Fig. 3.1 High-Level Architecture of the Recipe Finder System

The high-level architecture of the Recipe Finder system describes the major components of the application and the flow of information between the dataset, core DSA engine, algorithmic processing modules, and Java Swing graphical user interface. The architecture is designed to separate data processing, algorithm execution, and user interaction so that each component has a clear responsibility.

The Recipe Finder system begins with the recipe dataset stored in:

data/recipes.txt

The current dataset contains 100,000 recipe records. The dataset acts as the primary source of recipe information for the application. During initialization, the recipe records are loaded and

parsed by the core RecipeFinder.java component. The parser identifies the individual recipe records and extracts the required fields from each record.

The parsed recipe information is then stored in Java data structures so that it can be accessed by the different operations of the application. Storing the parsed records in memory avoids repeatedly reading and parsing the complete text file for every user request.

The central component of the architecture is RecipeFinder.java, which acts as the Core DSA Engine. It contains the main data-processing and algorithmic functionality of the project. The component is responsible for recipe loading, recipe parsing, searching, pattern matching, autocomplete preparation, fuzzy similarity, ingredient matching, greedy coverage, random recommendation, and parallel similarity processing. The documented component structure identifies these operations as part of RecipeFinder.java. Project Documentation format 20...

The search and autocomplete part of the architecture contains a Trie and multiple pattern-matching algorithms. The Trie is constructed using recipe names and is used to support prefix-based autocomplete. When a user enters a partial recipe name, the corresponding prefix is searched in the Trie and matching recipe suggestions can be provided.

The architecture also contains four pattern-matching techniques:

Naive Pattern Matching

Knuth-Morris-Pratt (KMP)

Rabin-Karp

Aho-Corasick

These algorithms provide different approaches for processing string patterns. The pattern-matching operations are integrated into the core DSA engine rather than being implemented as separate user-facing applications. This allows the project to demonstrate several string-processing techniques within the same Recipe Finder system.

Another major component is the Fuzzy and Similarity Search module. This component uses Levenshtein Edit Distance to compare the user's query with recipe names. The edit-distance calculation considers insertion, deletion, and substitution operations. The resulting distance is converted into a similarity percentage, allowing approximate matches to be identified when an exact search does not produce the required result.

The project also contains an Ingredient and Recommendation component. Ingredient-based processing uses bipartite matching to compare requested ingredients with recipe ingredients. This operation corresponds to the CO4 requirement of the project. The architecture also includes Greedy Set Cover for ingredient coverage, randomized recipe recommendation, and parallel similarity processing. These operations correspond to the higher-level recommendation and optimization functionality of the system. Project Documentation format 20...

The Java Swing GUI forms the user-facing layer of the architecture. It provides the interface through which users enter recipe queries, view autocomplete suggestions, perform ingredient-

based searches, request greedy coverage, and obtain random recipe recommendations. The GUI does not duplicate the DSA implementations. Instead, it interacts with the core `RecipeFinder.java` functionality and displays the resulting information to the user.

The GUI is implemented separately in:

`src/RecipeFinderGUI.java`

This separation between the GUI and the core DSA engine improves the organization of the application. Changes to the interface can be made without rewriting the underlying algorithms, while changes to an algorithm can be made without redesigning the entire graphical interface.

This architecture allows the same loaded recipe collection to support multiple operations. For example, the Trie can support prefix suggestions while pattern-matching methods process search patterns. Similarly, the same recipe objects can be used for ingredient matching, greedy coverage, and random recommendation.

The architecture also supports the project's scalability requirement by preparing the dataset and Trie during initialization rather than repeatedly rebuilding them during every user interaction. This is particularly relevant because the current system works with 100,000 recipe records.

3.2 Data and Component Description

3.2.1RecipeFinder.java

Contains:

- recipe loading
- recipe parsing
- pattern matching
- Trie
- exact search
- Levenshtein
- similarity
- ingredient matching
- greedy coverage
- random recommendation
- parallel fuzzy search

3.2.2RecipeFinderGUI.java

Contains:

- JFrame
- JPanel

- CardLayout
- JTextField
- JList
- JTextArea
- JButton
- JScrollPane
- DocumentListener

3.2.3 Recipe Dataset

data/recipes.txt

contains exactly:

100,000 recipe
records

Each record includes:

Recipe:

Ingredients:

Instructions:

Preparation Time:

.....

3.3 Search and Autocomplete Design

3.3.1 Trie-Based Autocomplete

The Trie is used for prefix search.

Example:

User enters:

bir

Possible suggestions:

Biryani

...

The Trie is built once from all recipe names.

The GUI uses **Document Listener** to update suggestions as the user types.

3.3.2 Exact Search

The actual implementation uses:

`exactSearch()`

It compares the query against recipe names using case-insensitive equality.

3.3.3 Pattern Matching

The matching recipe is then verified through:

`patternMatch()`

which calls:

Naive

KMP

Rabin-Karp

Aho-Corasick

3.3.4 Fuzzy Search

If exact search fails, the GUI uses:

`parallelFuzzySearch()`

Similarity is based on:

$$\text{Similarity} = (1 - \text{Edit Distance} / \text{Maximum String Length}) \times 100$$

The GUI uses a 70% similarity threshold for accepting a fuzzy result.

3.4 DSA and CO Mapping

CO	Implementation
CO1	Problem analysis, algorithm selection and complexity
CO2	Naive, KMP, Rabin-Karp, Aho-Corasick
CO3	Dynamic Programming, Levenshtein
CO4	Bipartite Matching
CO5	Greedy Set Cover
CO6	Randomized Recommendation + Parallel Similarity

3.5 Dataset Design

The Recipe Finder dataset is stored in data/recipes.txt. The current dataset contains exactly 100,000 recipe records. Each record begins with a Recipe: header and contains Ingredients:, Instructions:, Preparation Time:, and Cooking Time: fields. The parser identifies individual records using the Recipe: header.

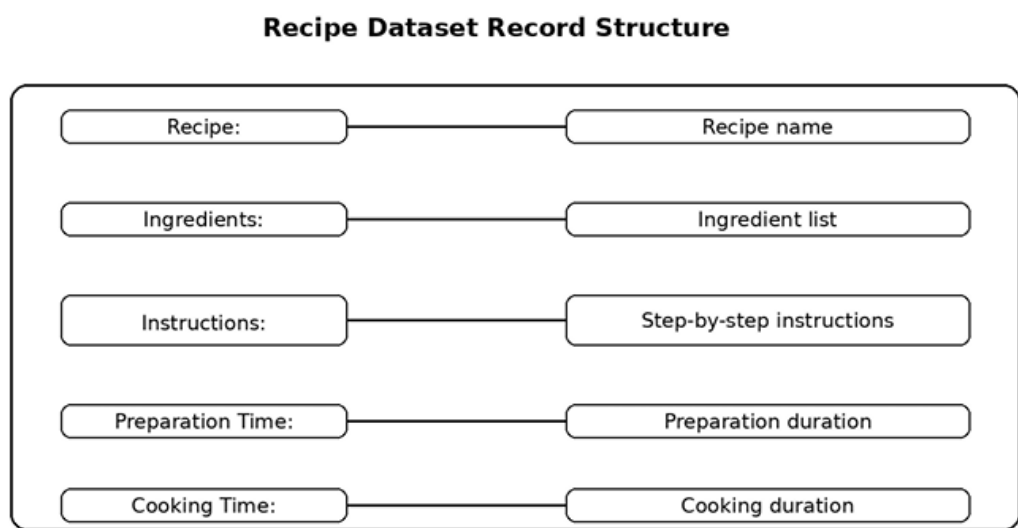


Fig. 3.4 Recipe Dataset Record Structure

CHAPTER 4 - IMPLEMENTATION

4.1 Module-Wise Implementation

Module 1 — Recipe Loading

Method:

loadRecipes()

Reads data/recipes.txt, identifies recipe headers, parses recipe blocks, and creates Recipe objects.

Module 2 — Ingredient Normalization

Method:

normalizeIngredient()

Converts ingredient text to lowercase and removes unnecessary characters for comparison.

Module 3 — Naive Pattern Matching

Method:

naivePatternMatch()

Compares the pattern at each possible position in the text.

Complexity:

$O(n \times m)$

Module 4 — KMP

Methods:

computeLPS()

kmpSearch()

KMP uses the Longest Prefix Suffix array.

Complexity:

$O(n + m)$

Module 5 — Rabin-Karp

Method:

`rabinKarpSearch()`

Uses:

hashing

rolling hash

character verification

Average:

$O(n + m)$

Worst:

$O(n \times m)$

Module 6 — Aho-Corasick

Method:

`ahoCorasickSearch()`

The implementation constructs a Trie-like automaton for the supplied pattern, builds failure links using a queue, and scans the text.

Important for your report: don't claim that your current implementation maintains a permanent multi-pattern dictionary. Your actual code constructs the automaton from the supplied pattern during verification.

Module 7 — Trie Autocomplete

Classes:

TrieNode
AutocompleteTrie

Methods:

buildAutocomplete()
getSuggestions()

The Trie supports prefix-based recipe suggestions.

Module 8 — Levenshtein Distance

Method:

levenshteinDistance()

Uses dynamic programming.

Operations:

insertion

deletion

substitution

Complexity:

$O(n \times m)$

Module 9 — Similarity

Method:

similarity()

Formula:

Similarity =
 $(1 - \text{Distance} / \text{Maximum Length}) \times 100$

Module 10 — Bipartite Matching

Methods:

bipartiteMatching()
tryMatch()

Requested ingredients form one side.

Recipe ingredients form the other side.

Matching edges are created when ingredients are equal after normalization.

Module 11 — Greedy Set Cover

Method:

greedySetCover()

The algorithm:

Stores requested ingredients.

Finds how many uncovered ingredients each recipe covers.

Selects the recipe with maximum coverage.

Removes covered ingredients.

Repeats until all are covered or no useful recipe remains.

Module 12 — Random Recommendation

Method:

randomizedRecommendation()

Selects a random index:

0 ... recipes.size()-1

and displays the corresponding recipe.

Module 13 — Parallel Similarity

Method:

parallelFuzzySearch()

Uses:

ExecutorService

Future

Independent similarity calculations are submitted to worker threads.

The best similarity result is selected.

4.2 PSEUDOCODE

Recipe Loading

```
LOAD_RECIPES(file)
```

```
    recipes ← empty list
```

```
    OPEN file
```

```
    recipeText ← empty
```

```
    FOR each line
```

```
        IF line starts with "Recipe:"
```

```
            AND recipeText is not empty
```

```
                recipe ← CREATE_RECIPE(recipeText)
```

```
                IF recipe is valid
```

```
                    add recipe to recipes
```

```
                recipeText ← empty
```

```
                append line to recipeText
```

```
    END FOR
```

```
    create final recipe if valid
```

```
BUILD_TRIE(recipes)
```

```
    create root
```

```
    FOR each recipe
```

```
        current  $\leftarrow$  root
```

```
        FOR each character in recipe name
```

```
            IF child does not exist
```

```
                create child
```

```
            current  $\leftarrow$  child
```

```
        add recipe to current node
```

```
    RETURN trie
```

Prefix Search

```
SEARCH_PREFIX(prefix)
```

```
    current ← root
```

```
    FOR each character in prefix
```

```
        IF child does not exist
```

```
            RETURN empty
```

```
        current ← child
```

```
    collect all recipes below current
```

```
    sort suggestions
```

```
    RETURN suggestions
```

Pattern Matching

```
PATTERN_MATCH(text, pattern)
```

```
    naive ← Naive(text, pattern)
```

```
    kmp ← KMP(text, pattern)
```

```
    rabin ← RabinKarp(text, pattern)
```

```
    aho ← AhoCorasick(text, pattern)
```

```
    IF all four results agree
```

```
        RETURN result
```

```
    RETURN false
```

Levenshtein

LEVENSHTTEIN(A, B)

create DP table

initialize first row

initialize first column

FOR i = 1 to length(A)

FOR j = 1 to length(B)

IF A[i-1] = B[j-1]

dp[i][j] = dp[i-1][j-1]

ELSE

dp[i][j] =

1 + minimum(

insertion,

deletion,

substitution

)

RETURN dp[n][m]

Bipartite Matching

```
BIPARTITE_MATCH(requested, recipeIngredients)
```

```
    matches  $\leftarrow$  0
```

```
    FOR each requested ingredient
```

```
        visited  $\leftarrow$  empty
```

```
        IF TRY_MATCH(requested ingredient)
```

```
            matches++
```

```
    RETURN matches
```

Greedy Set Cover

```
GREEDY_COVER(recipes, requested)
```

```
    uncovered  $\leftarrow$  requested
```

```
    selected  $\leftarrow$  empty
```

```
    WHILE uncovered is not empty
```

```
        bestRecipe  $\leftarrow$  null
```

```
        bestCoverage  $\leftarrow$  empty
```

```
        FOR each recipe
```

```
            coverage  $\leftarrow$  ingredients covered
```

```
                from uncovered
```



```
IF coverage > bestCoverage
```

```
    bestRecipe ← recipe
```

```
    bestCoverage ← coverage
```

```
IF no useful recipe exists
```

```
    BREAK
```

```
add bestRecipe to selected
```

```
remove bestCoverage from uncovered
```

```
RETURN selected
```

Random Recommendation

```
RANDOM_RECOMMEND(recipes)
```

```
    index ← random index
```

```
    RETURN recipes[index]
```

Parallel Fuzzy Search

```
PARALLEL_FUZZY_SEARCH(recipes, query)
```

```
    create thread pool
```

```
    FOR each recipe
```

```
        submit similarity calculation
```

```
    collect Future results
```

```
    select highest similarity
```

```
    shutdown thread pool
```

```
    RETURN best recipe
```

4.3 JAVA SWING INTERFACE

The Recipe Finder application provides a Java Swing graphical user interface for interacting with the implemented Data Structures and Algorithms. The interface provides a search-oriented environment through which users can access recipe search and recommendation functionality.

- The main GUI components include:
- recipe search field;
- autocomplete suggestion list;
- recipe result area;
- ingredient search field;
- greedy coverage input and result area;
- random recommendation area;
- status bar.

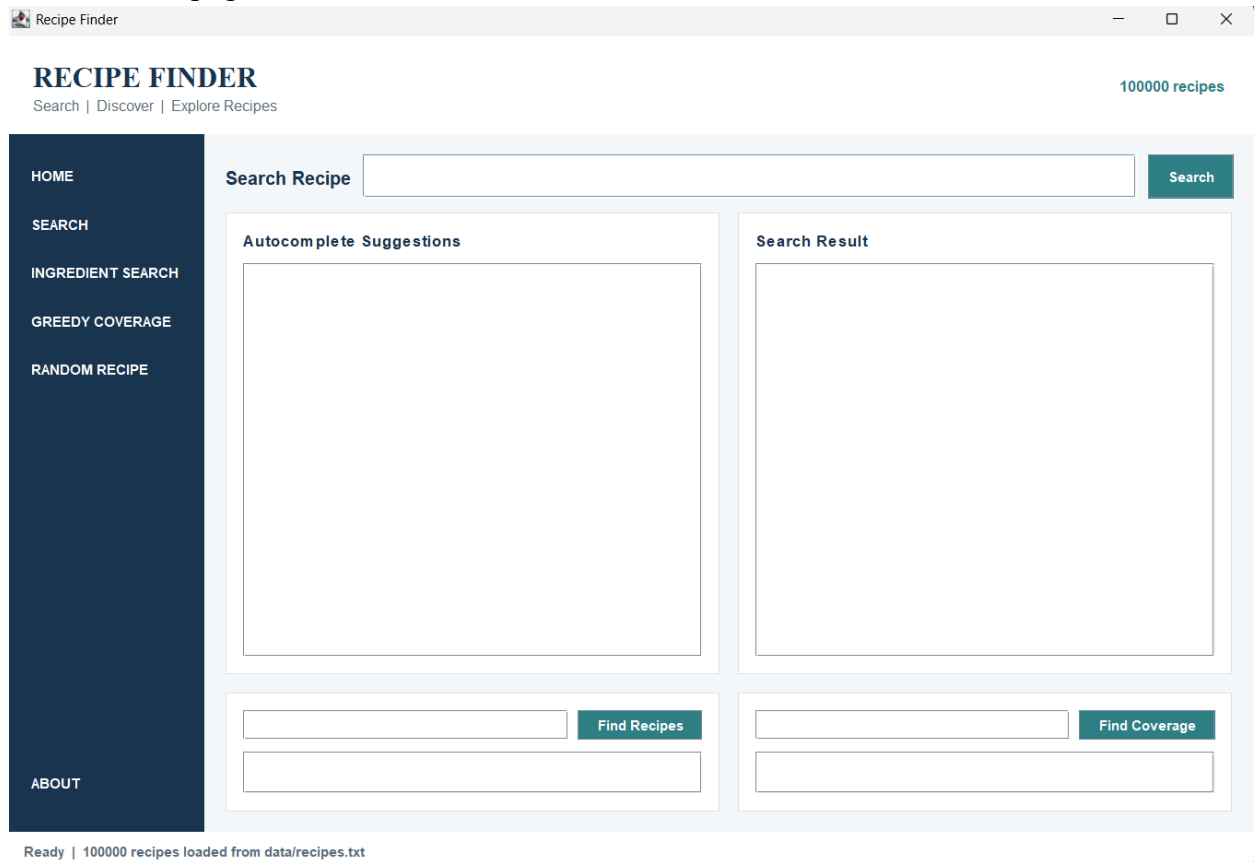
The graphical interface is implemented in `src/RecipeFinderGUI.java`, while the core DSA functionality remains in `src/RecipeFinder.java`. This separation allows the user interface to interact with the existing algorithm implementations without duplicating the underlying DSA logic.

- The GUI provides access to:
- Search Recipe
- Ingredient Search
- Greedy Coverage
- Random Recipe
- About

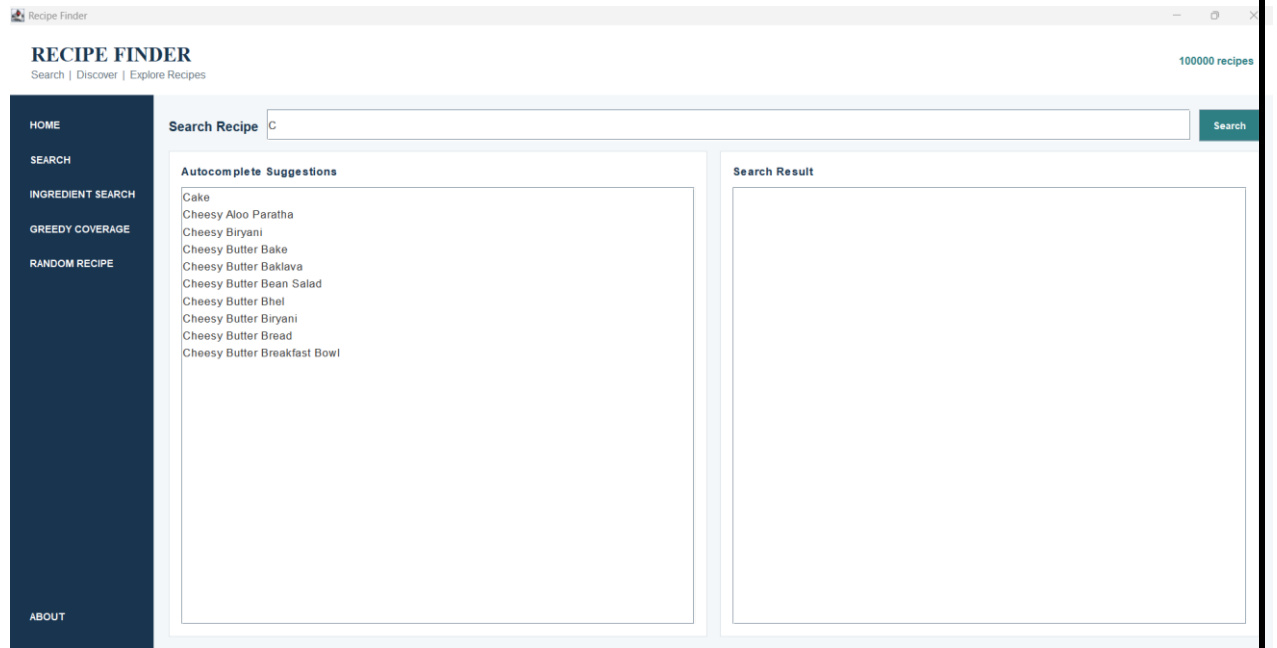
The GUI also contains:

- search field
- autocomplete list
- result area
- ingredient field
- coverage field
- random recommendation area
- status bar

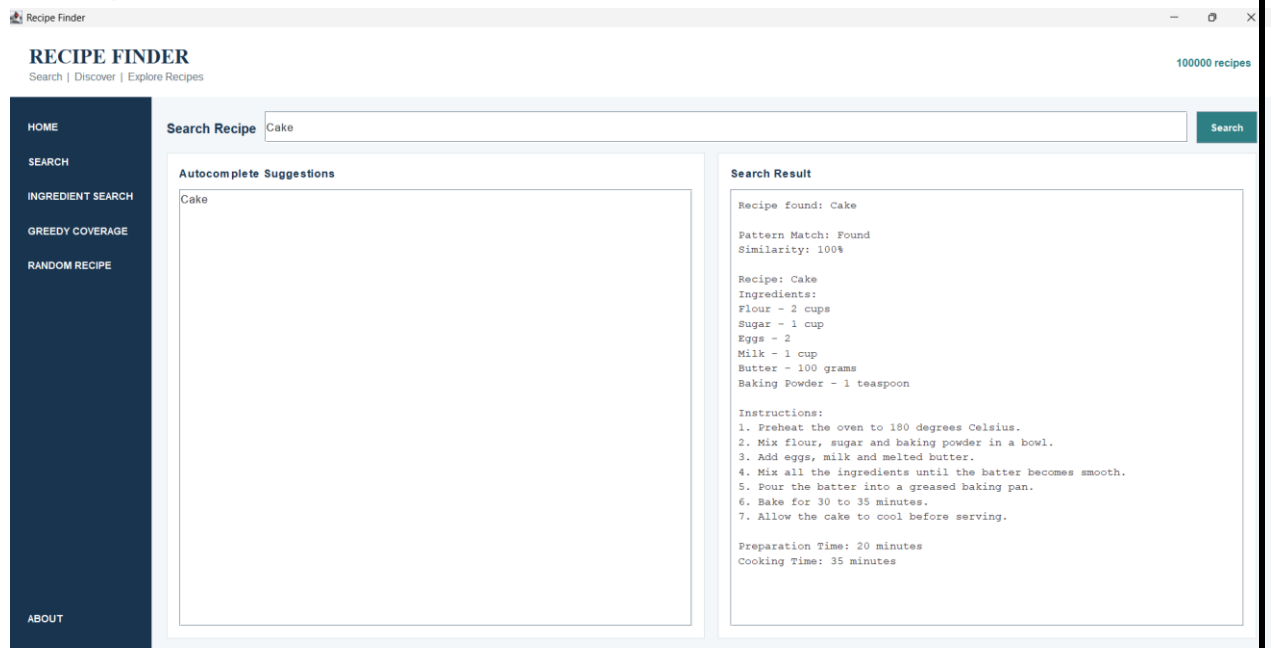
01-dashboard.png



02-search.png

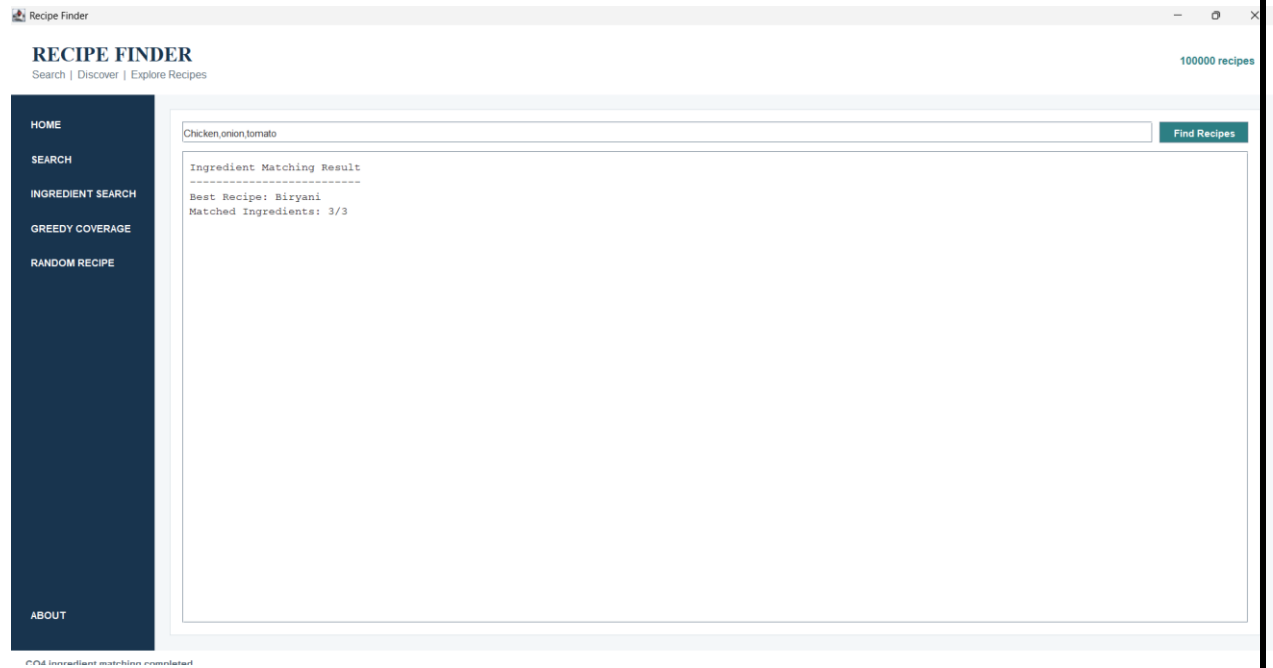


Please enter a recipe name.

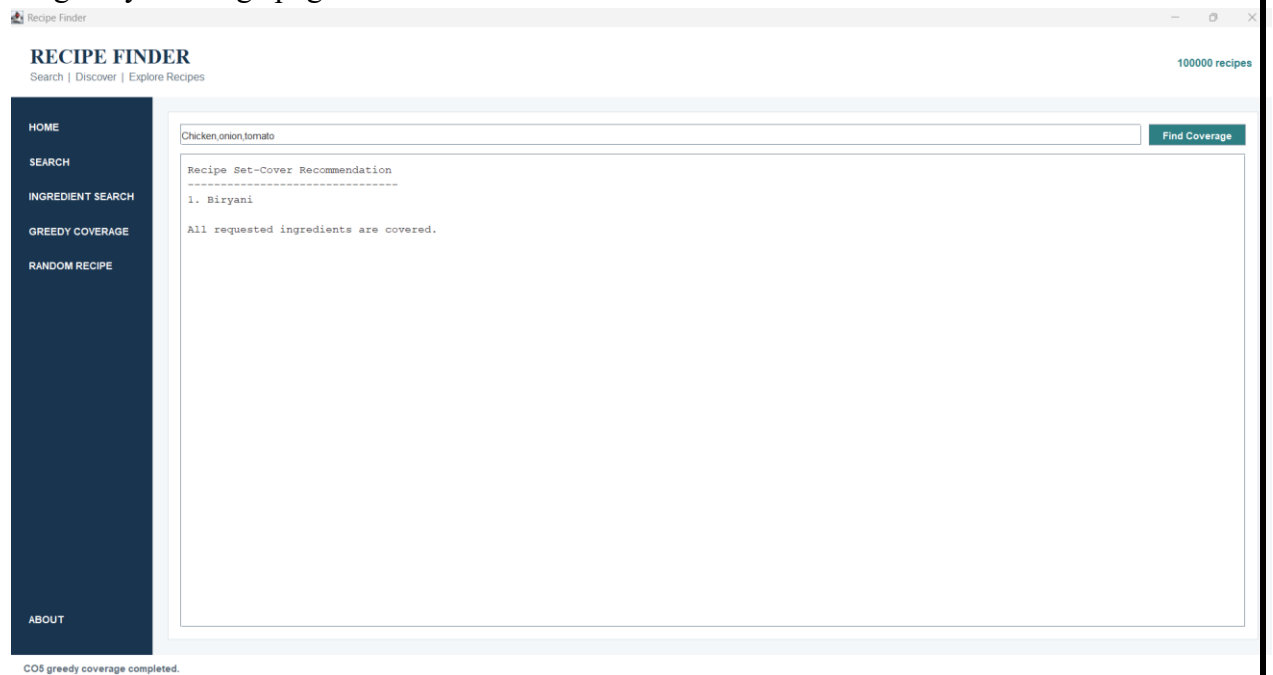


Recipe found using exact pattern matching.

03-ingredient-search.png



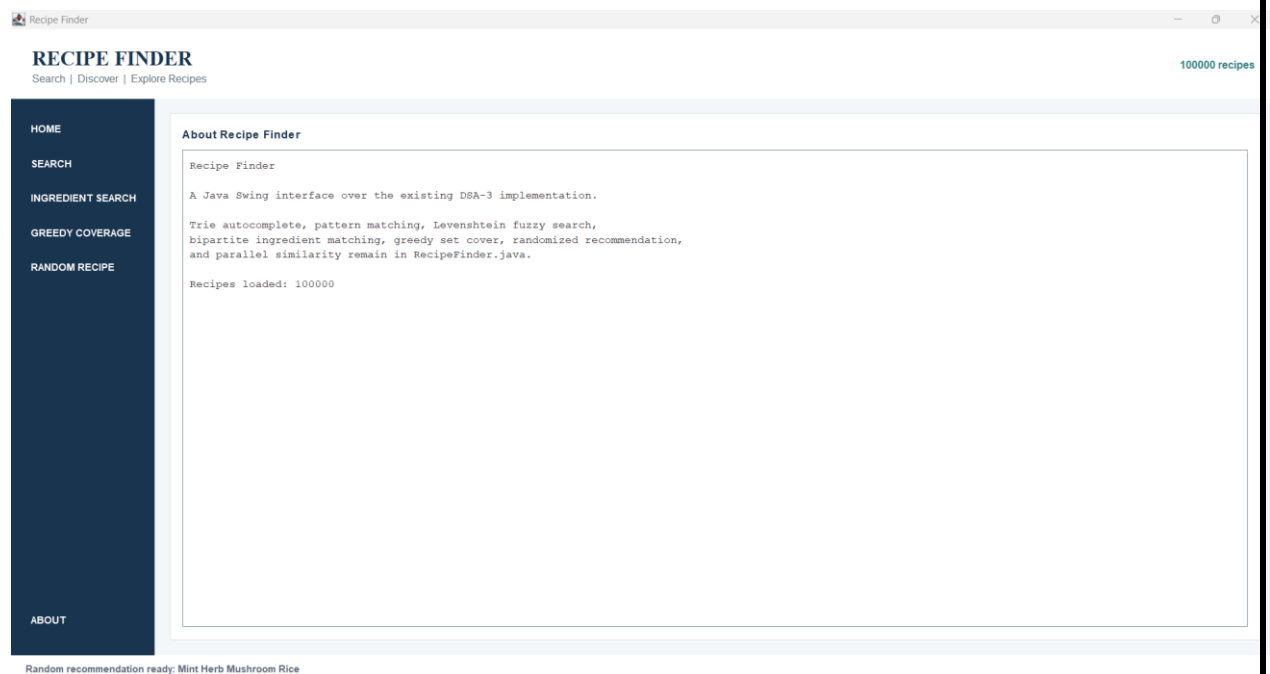
04-greedy-coverage.png



05-random-recipe.png



06-about.png



CHAPTER 5 - RESULTS

5.1 Testing Strategy

Test the application using:

Test	Input	Expected Result
Exact Search	Cake	Cake recipe
Case-Insensitive	CAKE	Cake identified
Prefix	c	Suggestions
Prefix	bir	Biryani suggestions
Fuzzy	cak	Closest recipe
Ingredient	chicken,onion,tomato	Ingredient result
Coverage	chicken,onion,tomato	Greedy coverage
Random	random	Random recipe
Empty	blank	Validation
Invalid	Burger	Not found

5.2 Functional Results

5.2.1 Exact Search

Input: Cake

Result: Recipe found: Cake.

The complete recipe information, including ingredients, instructions, preparation time and cooking time, is displayed.

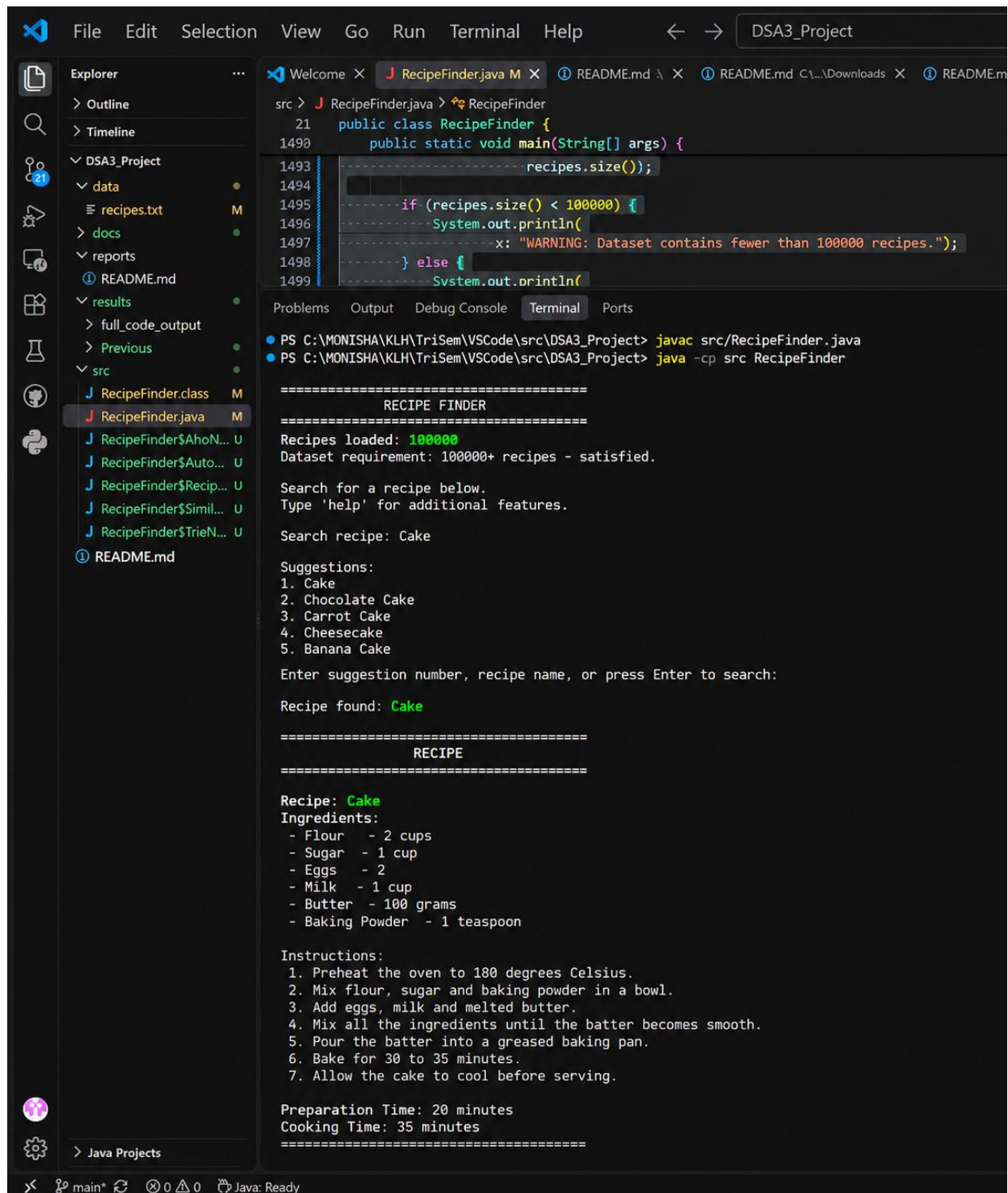


Fig. 5.1 Exact Recipe Search Execution

5.2.2 Case-Insensitive

Input: CAKE

Result: The system identifies the recipe despite the difference in capitalization.

```
src > J RecipeFinder.java > RecipeFinder
21 public class RecipeFinder {
1460 public static void main(String[] args) {
1493     ... recipes.size());
1494
1495     if (recipes.size() < 2000) {
1496         System.out.println(
1497             ... "WARNING: Dataset contains fewer than 2000 recipes.");
```

```
PS C:\MONISHA\KLH\TriSem\VSCode\src\DSA3_Project> java -cp src RecipeFinder

Search recipe: CAKE

Suggestions:
1. Cake

Enter suggestion number, recipe name, or press Enter to search: 1

Selected Recipe: Cake

=====
RECIPE
=====

Recipe: Cake
Ingredients:
Flour - 2 cups
Sugar - 1 cup
Eggs - 2
Milk - 1 cup
Butter - 100 grams
Baking Powder - 1 teaspoon

Instructions:
1. Preheat the oven to 180 degrees Celsius.
2. Mix flour, sugar and baking powder in a bowl.
3. Add eggs, milk and melted butter.
4. Mix all the ingredients until the batter becomes smooth.
5. Pour the batter into a greased baking pan.
6. Bake for 30 to 35 minutes.
7. Allow the cake to cool before serving.

Preparation Time: 20 minutes
Cooking Time: 35 minutes
=====

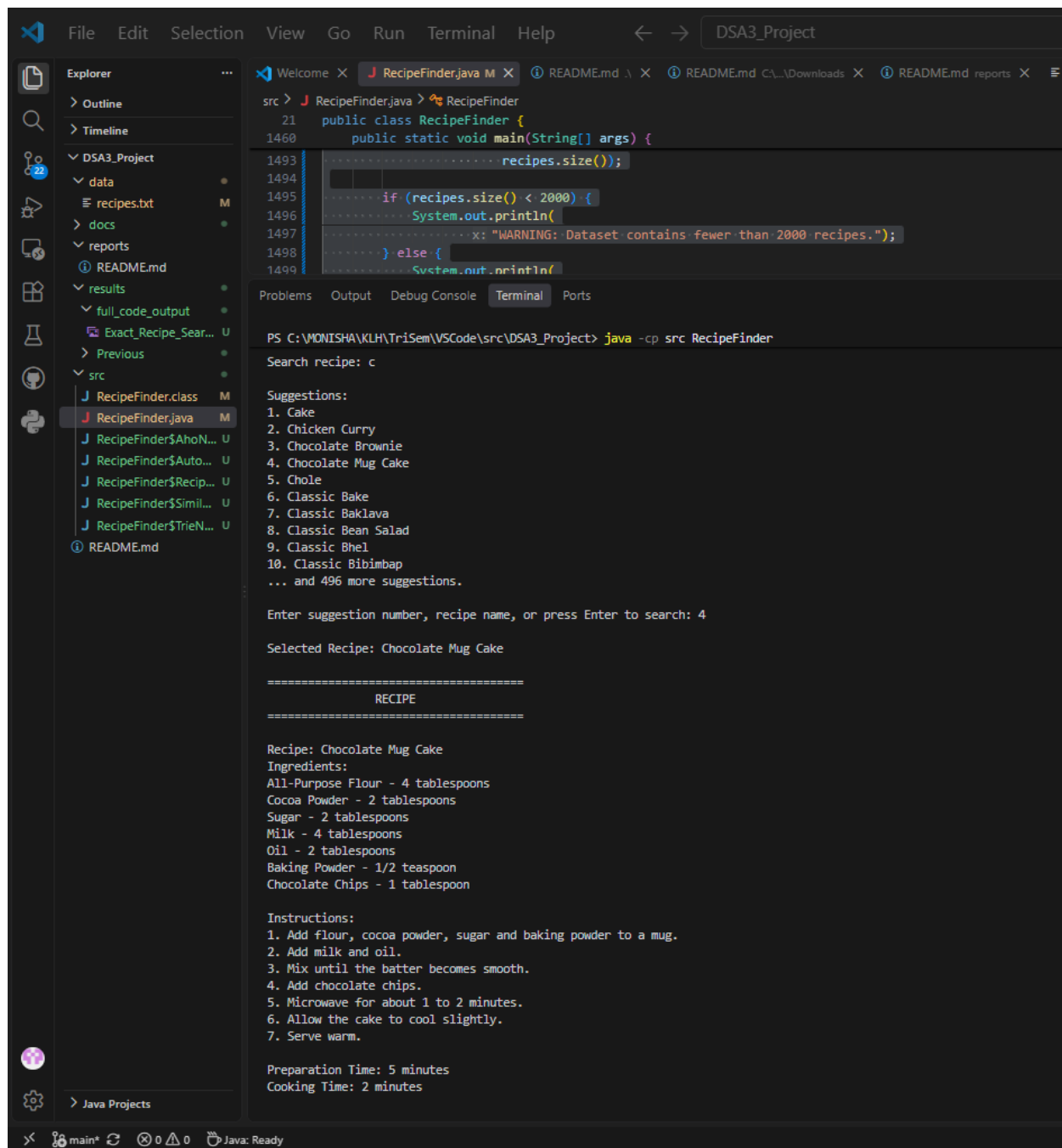
Search recipe: 
```

Fig. 5.2 Case-Insensitive Recipe Search

5.2.3 Autocomplete

Input: c

Result: Trie-based suggestions are displayed.



```
src > J RecipeFinder.java > RecipeFinder
21 public class RecipeFinder {
1460 public static void main(String[] args) {
1493 ..... recipes.size());
1494 .....
1495 ..... if (recipes.size() < 2000) {
1496 ..... System.out.println(
1497 ..... x: "WARNING: Dataset contains fewer than 2000 recipes.");
1498 ..... } else {
1499 ..... System.out.println(
```

PS C:\MONISHA\KLH\TriSem\VSCode\src\DSA3_Project> java -cp src RecipeFinder

Search recipe: c

Suggestions:

1. Cake
2. Chicken Curry
3. Chocolate Brownie
4. Chocolate Mug Cake
5. Chole
6. Classic Bake
7. Classic Baklava
8. Classic Bean Salad
9. Classic Bhel
10. Classic Bibimbap

... and 496 more suggestions.

Enter suggestion number, recipe name, or press Enter to search: 4

Selected Recipe: Chocolate Mug Cake

=====

RECIPE

=====

Recipe: Chocolate Mug Cake

Ingredients:

- All-Purpose Flour - 4 tablespoons
- Cocoa Powder - 2 tablespoons
- Sugar - 2 tablespoons
- Milk - 4 tablespoons
- Oil - 2 tablespoons
- Baking Powder - 1/2 teaspoon
- Chocolate Chips - 1 tablespoon

Instructions:

1. Add flour, cocoa powder, sugar and baking powder to a mug.
2. Add milk and oil.
3. Mix until the batter becomes smooth.
4. Add chocolate chips.
5. Microwave for about 1 to 2 minutes.
6. Allow the cake to cool slightly.
7. Serve warm.

Preparation Time: 5 minutes

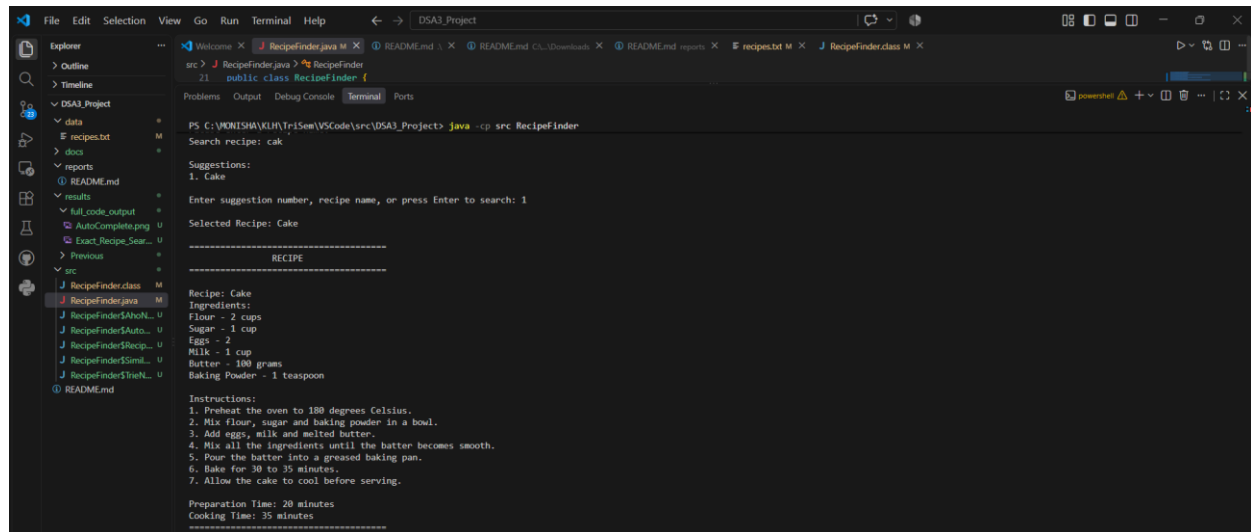
Cooking Time: 2 minutes

Fig. 5.3 Trie-Based Autocomplete

5.2.4 Fuzzy Search

Input: cak

Result: The system searches for the closest recipe using Levenshtein similarity.



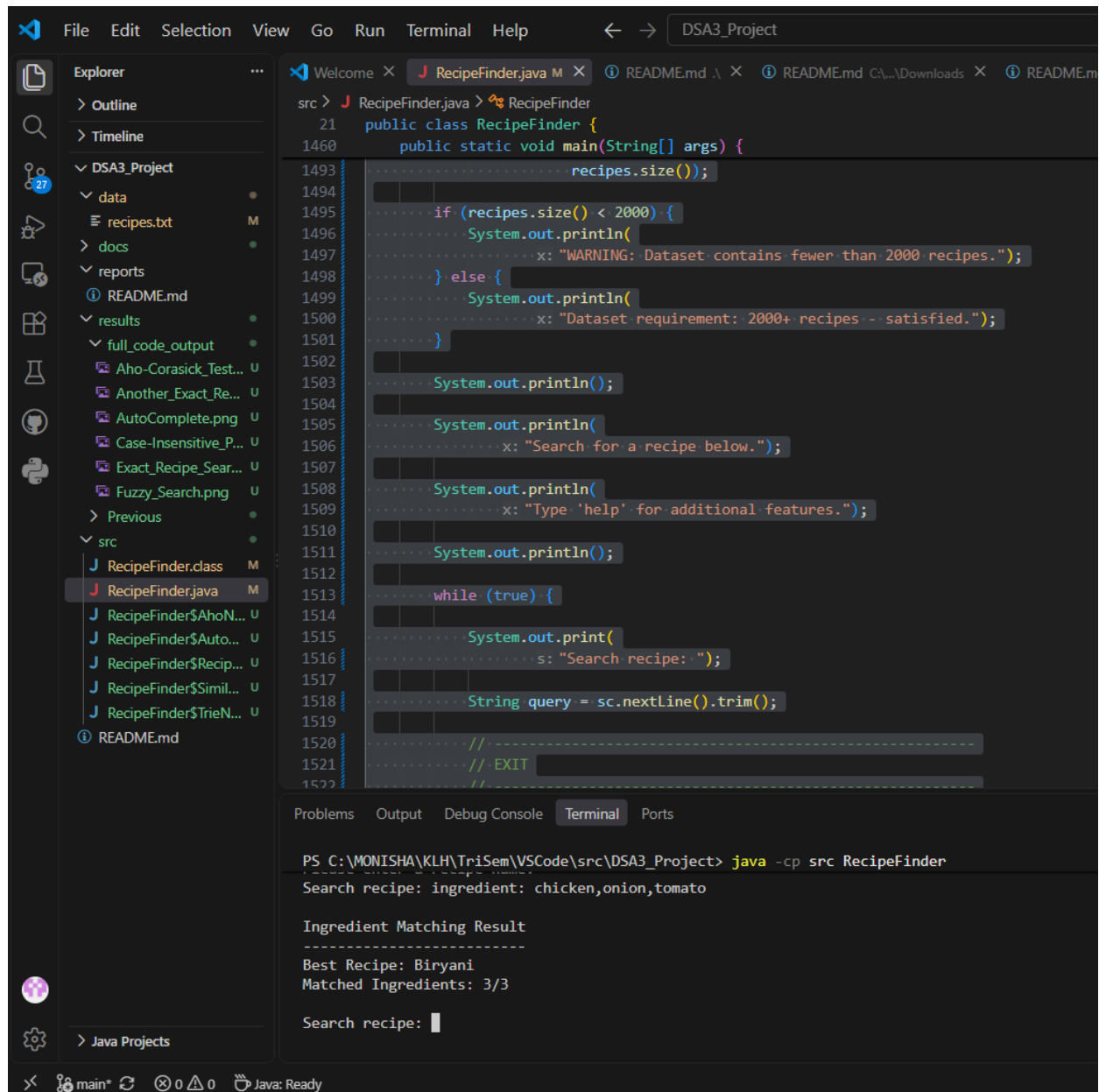
```
PS C:\WORK\ISHA\KLH\Tr1See\VSCode\src\DSA3_Project> java -cp src RecipeFinder
Search recipe: cak
Suggestions:
1. Cake
Enter suggestion number, recipe name, or press Enter to search: 1
Selected Recipe: Cake
=====
RECIPE
=====
Recipe: Cake
Ingredients:
Flour - 2 cups
Sugar - 1 cup
Eggs - 2
Milk - 1 cup
Butter - 100 grams
Baking Powder - 1 teaspoon
Instructions:
1. Preheat the oven to 180 degrees Celsius.
2. Mix flour, sugar and baking powder in a bowl.
3. Add eggs, milk and melted butter.
4. Mix all the ingredients until the batter becomes smooth.
5. Pour the batter into a greased baking pan.
6. Bake for 30 to 35 minutes.
7. Allow the cake to cool before serving.
Preparation Time: 20 minutes
Cooking Time: 35 minutes
=====
```

Fig. 5.4 Fuzzy Search Using Levenshtein Similarity

5.2.5 Ingredient Search

Input: chicken, onion, tomato

Result: The application performs ingredient matching.



```
File Edit Selection View Go Run Terminal Help
DSA3_Project

Welcome x J RecipeFinder.java x README.md x README.md CA...\Downloads x README.m

src > J RecipeFinder.java > RecipeFinder
21 public class RecipeFinder {
1460 public static void main(String[] args) {
1493     recipes.size());
1494
1495     if (recipes.size() < 2000) {
1496         System.out.println(
1497             x: "WARNING: Dataset contains fewer than 2000 recipes.");
1498     } else {
1499         System.out.println(
1500             x: "Dataset requirement: 2000+ recipes -- satisfied.");
1501     }
1502     System.out.println();
1503
1504     System.out.println(
1505         x: "Search for a recipe below.");
1506
1507     System.out.println(
1508         x: "Type 'help' for additional features.");
1509
1510     System.out.println();
1511
1512     while (true) {
1513         System.out.print(
1514             s: "Search recipe: ");
1515
1516         String query = sc.nextLine().trim();
1517
1518         //-----
1519         //EXIT
1520         //-----
1521
1522     }
1523 }
```

Problems Output Debug Console Terminal Ports

```
PS C:\MONISHA\KLH\TriSem\VSCode\src\DSA3_Project> java -cp src RecipeFinder
Search recipe: ingredient: chicken,onion,tomato

Ingredient Matching Result
-----
Best Recipe: Biryani
Matched Ingredients: 3/3

Search recipe: 
```

Fig. 5.5 CO4 Ingredient Matching Using Bipartite Matching

5.2.6 Greedy Coverage

Input: chicken, onion, tomato

Result: The application selects recipes according to currently uncovered ingredient coverage.

```
File Edit Selection View Go Run Terminal Help DSA3_Project
src > J RecipeFinder.java M X README.md \ X README.md C:\...\Downloads X README
21 public class RecipeFinder {
1460 public static void main(String[] args) {
1493     recipes.size());
1494
1495     if (recipes.size() < 2000) {
1496         System.out.println(
1497             x: "WARNING: Dataset contains fewer than 2000 recipes.");
1498     } else {
1499         System.out.println(
1500             x: "Dataset requirement: 2000+ recipes -- satisfied.");
1501     }
1502
1503     System.out.println();
1504
1505     System.out.println(
1506         x: "Search for a recipe below.");
1507
1508     System.out.println(
1509         x: "Type 'help' for additional features.");
1510
1511     System.out.println();
1512
1513     while (true) {
1514
1515         System.out.print(
1516             s: "Search recipe: ");
1517
1518         String query = sc.nextLine().trim();
1519
Problems Output Debug Console Terminal Ports
PS C:\MONISHA\KLH\TriSem\VSCode\src\DSA3_Project> java -cp src RecipeFinder
Search recipe: cover: chicken,onion,tomato

Recipe Set-Cover Recommendation
-----
1. Biryani

All requested ingredients are covered.

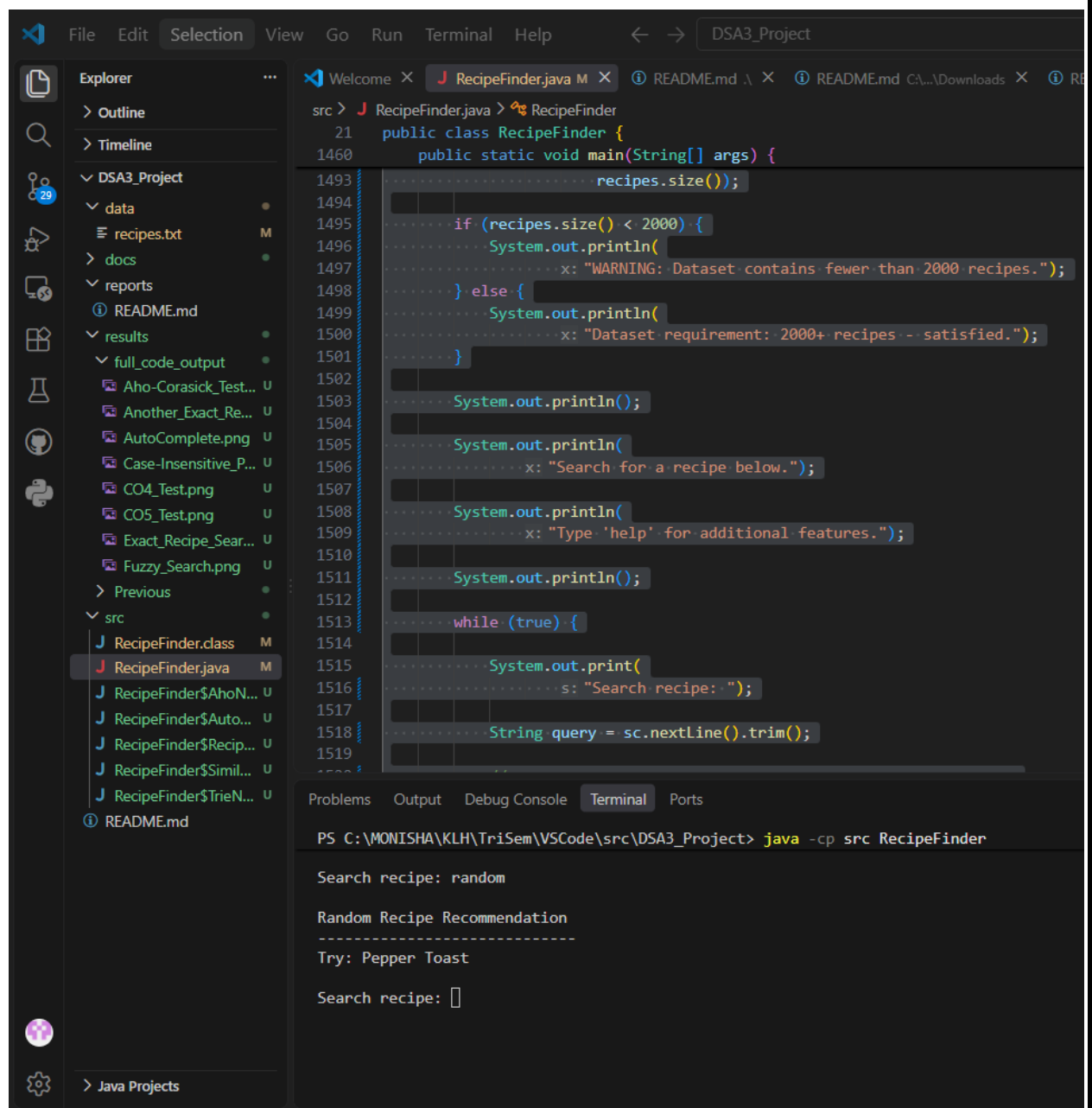
Search recipe: 
```

Fig. 5.6 CO5 Greedy Set Cover Execution

5.2.7 Random

Input: random

Result: The application selects a random recipe.



The screenshot displays the Visual Studio Code editor interface. The Explorer panel on the left shows the project structure for 'DSA3_Project', with the 'src' folder expanded and 'RecipeFinder.java' selected. The main editor window shows the code for 'RecipeFinder.java', which includes a 'main' method that checks the size of the 'recipes' dataset and prints a warning if it's below 2000. The code also includes a loop for searching recipes. The Terminal panel at the bottom shows the command 'java -cp src RecipeFinder' being executed, and the output displays the search results for the input 'random', showing a 'Random Recipe Recommendation' of 'Pepper Toast'.

```
src > J RecipeFinder.java > RecipeFinder
21 public class RecipeFinder {
1460 public static void main(String[] args) {
1493 ..... recipes.size());
1494 .....
1495 ..... if (recipes.size() < 2000) {-{
1496 ..... System.out.println(
1497 ..... x: "WARNING: Dataset contains fewer than 2000 recipes.");
1498 ..... } else {-{
1499 ..... System.out.println(
1500 ..... x: "Dataset requirement: 2000+ recipes - satisfied.");
1501 ..... }
1502 ..... System.out.println();
1503 .....
1504 ..... System.out.println(
1505 ..... x: "Search for a recipe below.");
1506 .....
1507 ..... System.out.println(
1508 ..... x: "Type 'help' for additional features.");
1509 .....
1510 ..... System.out.println();
1511 .....
1512 ..... while (true) {-{
1513 .....
1514 ..... System.out.print(
1515 ..... s: "Search recipe: ");
1516 .....
1517 ..... String query = sc.nextLine().trim();
1518 .....
1519 ..... }
```

```
PS C:\MONISHA\KLH\TriSem\VSCode\src\DSA3_Project> java -cp src RecipeFinder

Search recipe: random

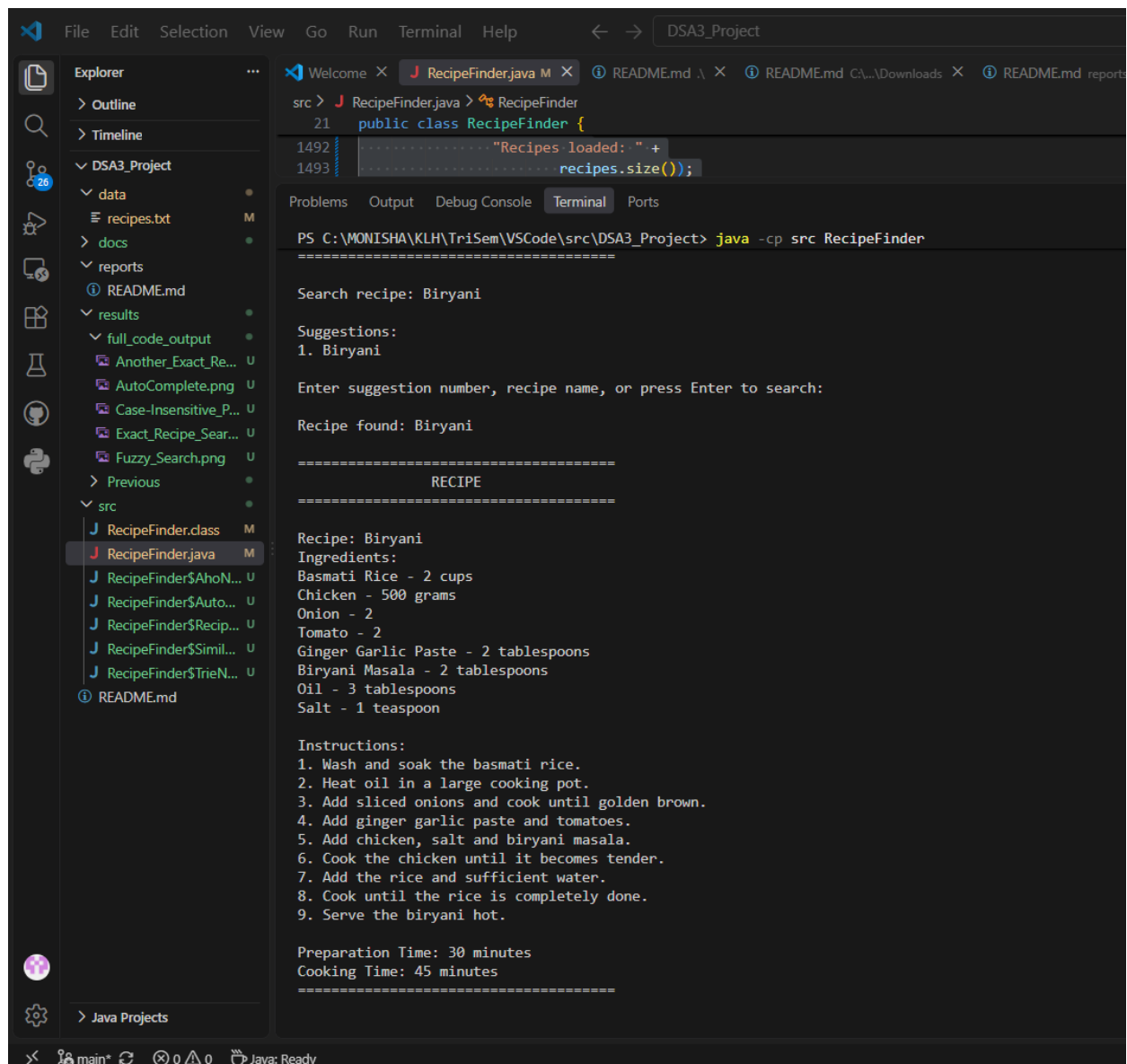
Random Recipe Recommendation
-----
Try: Pepper Toast

Search recipe: 
```

Fig. 5.7 CO6 Random Recipe Recommendation

5.2.8 Aho-Corasick Pattern Matching

The Aho-Corasick pattern-matching routine is used internally during pattern verification. The test confirms that the implemented routine processes the supplied pattern successfully.



The screenshot shows the Visual Studio Code interface with a project named 'DSA3_Project'. The Explorer panel on the left shows the project structure, including a 'src' directory with 'RecipeFinder.class' and 'RecipeFinder.java'. The main editor displays the 'RecipeFinder.java' file with the following code:

```
src > J RecipeFinder.java > RecipeFinder
21 public class RecipeFinder {
1492 ..... "Recipes loaded: " +
1493 ..... recipes.size();
```

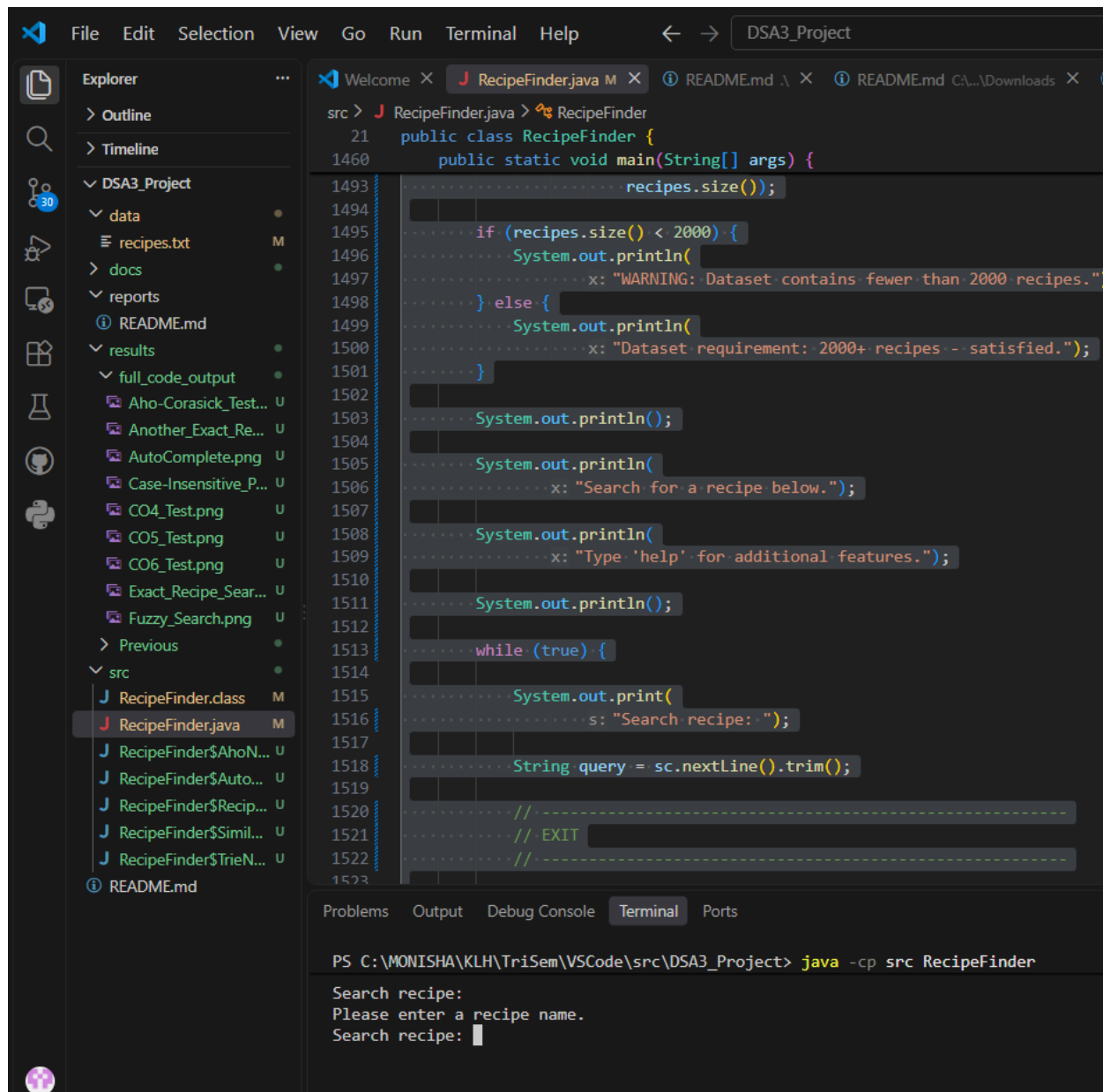
The Terminal panel at the bottom shows the command `java -cp src RecipeFinder` being executed. The output of the program is as follows:

```
PS C:\MONISHA\KLH\TriSem\VSCode\src\DSA3_Project> java -cp src RecipeFinder
=====
Search recipe: Biryani
Suggestions:
1. Biryani
Enter suggestion number, recipe name, or press Enter to search:
Recipe found: Biryani
=====
RECIPE
=====
Recipe: Biryani
Ingredients:
Basmati Rice - 2 cups
Chicken - 500 grams
Onion - 2
Tomato - 2
Ginger Garlic Paste - 2 tablespoons
Biryani Masala - 2 tablespoons
Oil - 3 tablespoons
Salt - 1 teaspoon
Instructions:
1. Wash and soak the basmati rice.
2. Heat oil in a large cooking pot.
3. Add sliced onions and cook until golden brown.
4. Add ginger garlic paste and tomatoes.
5. Add chicken, salt and biryani masala.
6. Cook the chicken until it becomes tender.
7. Add the rice and sufficient water.
8. Cook until the rice is completely done.
9. Serve the biryani hot.
Preparation Time: 30 minutes
Cooking Time: 45 minutes
=====
```

Fig. 5.8 Aho-Corasick Pattern Matching Execution

5.2.9 Empty Input Validation

The application was tested with an empty search input. The system displays a validation message instead of terminating unexpectedly.



The screenshot shows the Visual Studio Code editor with the 'RecipeFinder.java' file open. The file contains a Java program that checks the size of a dataset and provides a search interface. The terminal at the bottom shows the command to run the program, which prompts the user to enter a recipe name. The input field is currently empty, demonstrating the validation scenario.

```
src > J RecipeFinder.java > RecipeFinder
21 public class RecipeFinder {
1460 public static void main(String[] args) {
1493 ..... recipes.size());
1494 .....
1495 ..... if (recipes.size() < 2000) {
1496 ..... System.out.println(
1497 ..... x: "WARNING: Dataset contains fewer than 2000 recipes.")
1498 ..... } else {
1499 ..... System.out.println(
1500 ..... x: "Dataset requirement: 2000+ recipes - satisfied.");
1501 ..... }
1502 .....
1503 ..... System.out.println();
1504 .....
1505 ..... System.out.println(
1506 ..... x: "Search for a recipe below.");
1507 .....
1508 ..... System.out.println(
1509 ..... x: "Type 'help' for additional features.");
1510 .....
1511 ..... System.out.println();
1512 .....
1513 ..... while (true) {
1514 .....
1515 ..... System.out.print(
1516 ..... s: "Search recipe:");
1517 .....
1518 ..... String query = sc.nextLine().trim();
1519 .....
1520 ..... //-----
1521 ..... // EXIT
1522 ..... //-----
1523 .....
```

Problems Output Debug Console Terminal Ports

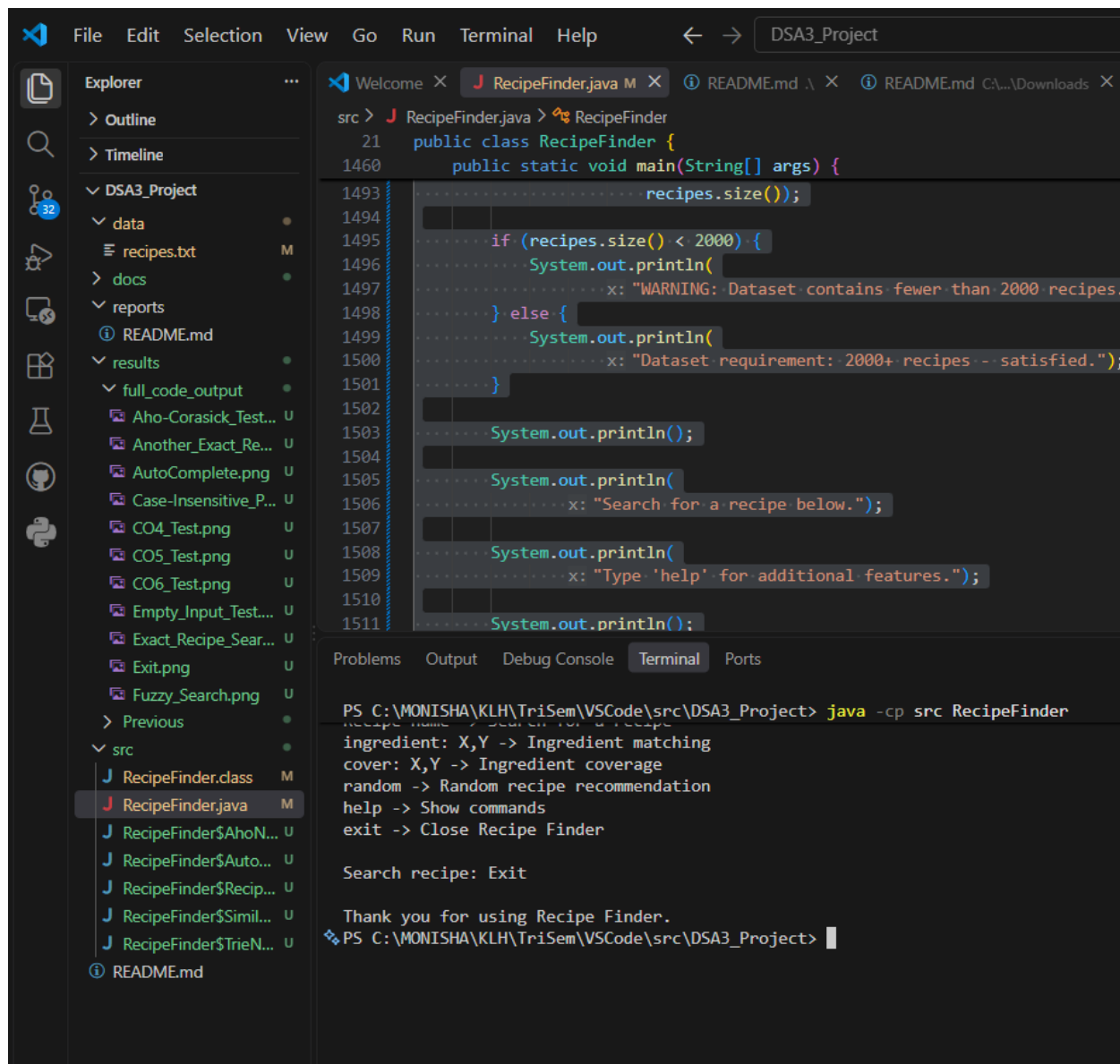
PS C:\MONISHA\KLH\TriSem\VSCode\src\DSA3_Project> java -cp src RecipeFinder

Search recipe:
Please enter a recipe name.
Search recipe:

Fig. 5.9 Empty Input Validation

5.2.10 Exit Operation

The exit operation was tested to verify that the application terminates normally when the user chooses to close the Recipe Finder application.



The screenshot displays the Visual Studio Code interface with the 'RecipeFinder.java' file open. The file contains a Java class with a 'main' method that checks the size of a 'recipes' array and prints various messages based on the size. The terminal at the bottom shows the command 'java -cp src RecipeFinder' being executed, followed by the application's output, which includes a list of commands (ingredient, cover, random, help, exit) and a final 'Thank you for using Recipe Finder.' message.

```
src > RecipeFinder.java > RecipeFinder
21 public class RecipeFinder {
1460 public static void main(String[] args) {
1493     recipes.size());
1494
1495     if (recipes.size() < 2000) {
1496         System.out.println(
1497             x: "WARNING: Dataset contains fewer than 2000 recipes.
1498         } else {
1499             System.out.println(
1500                 x: "Dataset requirement: 2000+ recipes -- satisfied.");
1501         }
1502
1503         System.out.println();
1504
1505         System.out.println(
1506             x: "Search for a recipe below.");
1507
1508         System.out.println(
1509             x: "Type 'help' for additional features.");
1510
1511         System.out.println();
1512     }
1513 }
```

```
PS C:\MONISHA\KLH\TriSem\VSCode\src\DSA3_Project> java -cp src RecipeFinder
recipe name -> Search for a recipe
ingredient: X,Y -> Ingredient matching
cover: X,Y -> Ingredient coverage
random -> Random recipe recommendation
help -> Show commands
exit -> Close Recipe Finder

Search recipe: Exit

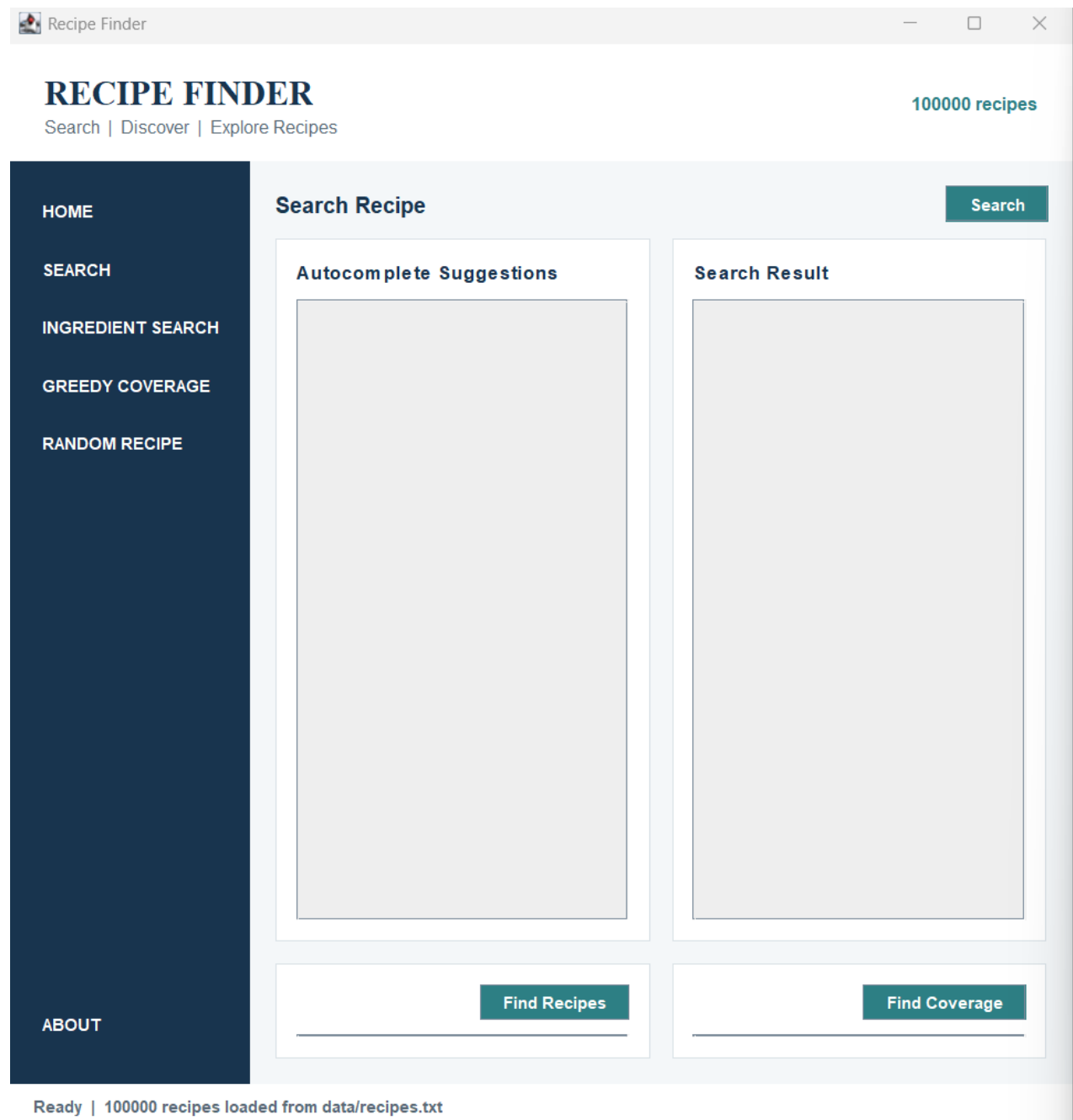
Thank you for using Recipe Finder.
PS C:\MONISHA\KLH\TriSem\VSCode\src\DSA3_Project>
```

Fig. 5.10 Application Exit Operation

5.3 GUI Results

Figure 5.11

results/GUI_OUTPUT/01-dashboard.png



Caption:

Fig. 5.11 Java Swing Recipe Finder Dashboard

Figure 5.12

02-search.png

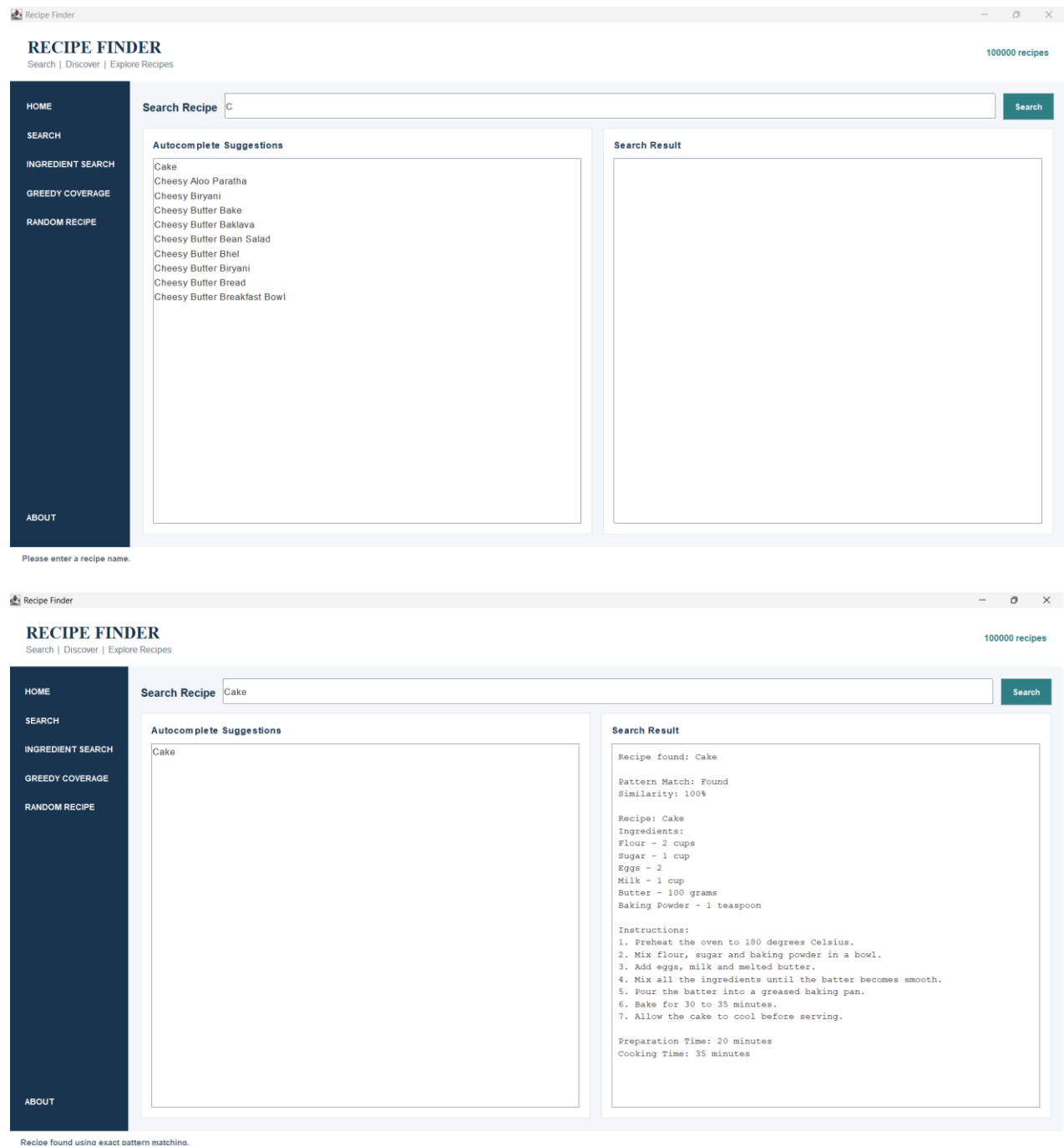


Fig. 5.12 Recipe Search and Autocomplete

Figure 5.13

03-ingredient-search.png

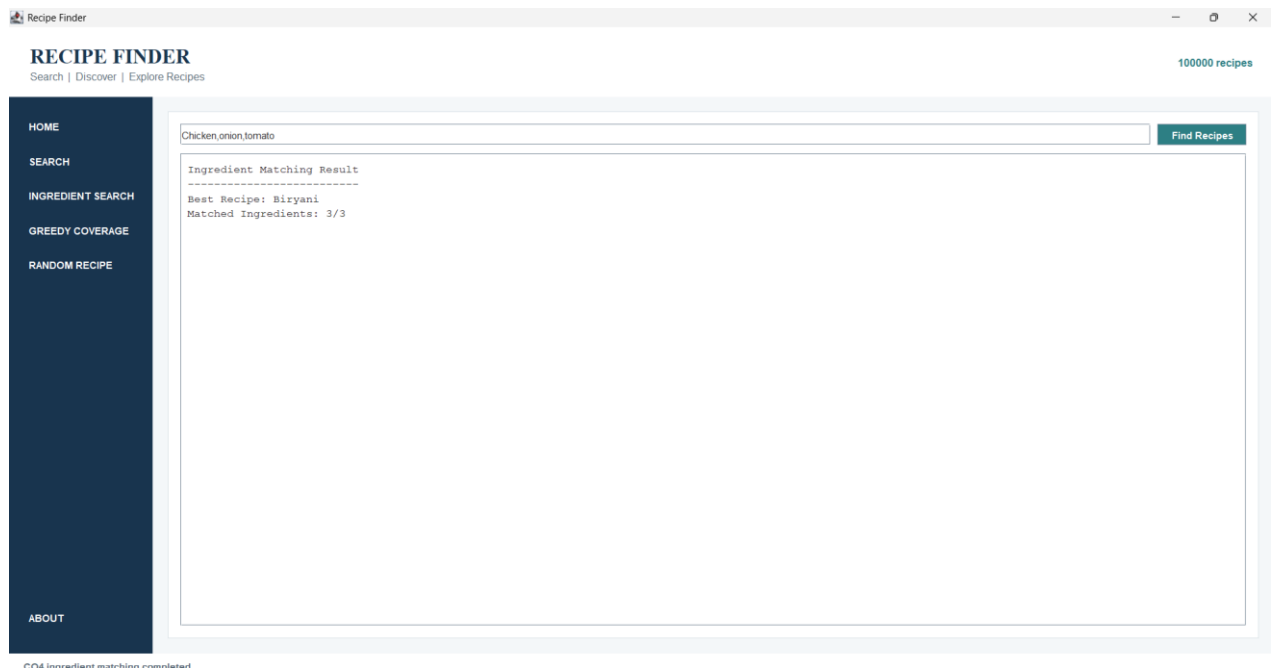


Fig. 5.13 CO4 Ingredient Search

Figure 5.14

04-greedy-coverage.png

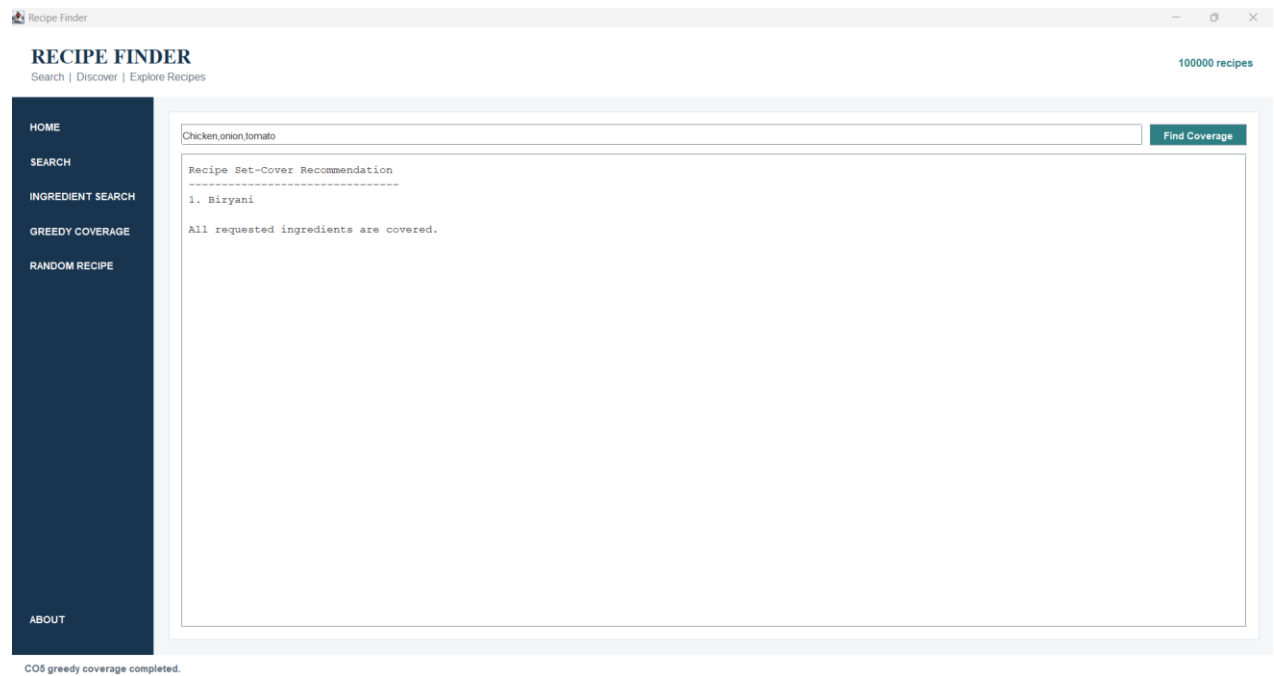


Fig. 5.14 CO5 Greedy Coverage

Figure 5.15

05-random-recipe.png

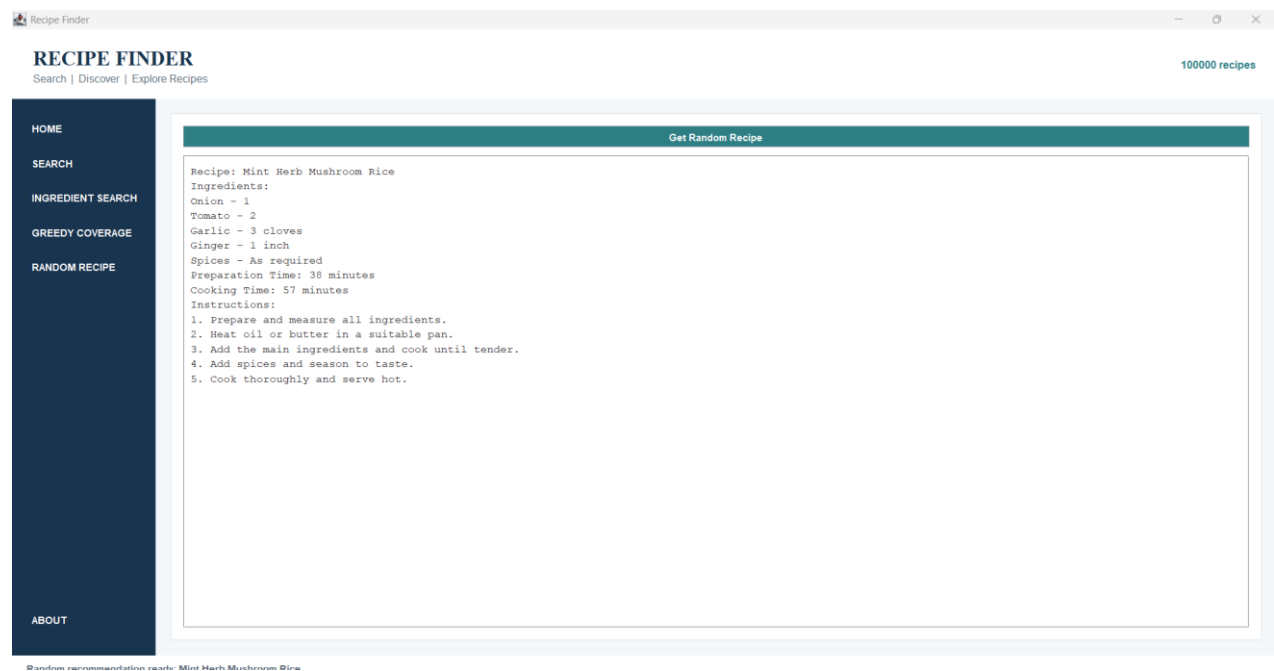


Fig. 5.15 CO6 Random Recommendation

5.4 COMPLEXITY ANALYSIS

Algorithm	Complexity
Naïve Pattern Matching	$O(n \times m)$ worst case
KMP	$O(n + m)$
Rabin-Karp	$O(n + m)$ average
Rabin-Karp	$O(n \times m)$ worst case
Aho-Corasick	$O(P + N + Z)$
Trie Prefix Search	Approximately $O(p + k)$
Levenshtein	$O(n \times m)$
Bipartite Matching	Depends on requested and recipe ingredient counts
Greedy Coverage	Depends on recipes and requested ingredients
Random Recommendation	$O(1)$ after loading
Parallel Fuzzy Search	Distributed independent similarity calculations

Where:

n = text length

m = pattern length

P = total pattern length

N = text length

Z = number of matches

p = prefix length

k = number of suggestions

CHAPTER 6 — CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

During the development of the Recipe Finder system, several technical and implementation-related challenges were encountered. The project combines multiple Data Structures and Algorithms with a large recipe dataset and a Java Swing graphical interface. Therefore, the main challenges were related to integrating different algorithms, handling the increased dataset size, maintaining efficient autocomplete, parsing structured recipe records, performing fuzzy and ingredient-based searches, and connecting the existing DSA implementation with the GUI.

6.1.1 Integrating Multiple Pattern-Matching Algorithms

The project required several pattern-matching algorithms to operate within a single recipe-search application. The implemented pattern-matching techniques include Naive Pattern Matching, Knuth-Morris-Pratt (KMP), Rabin-Karp and Aho-Corasick. Each algorithm follows a different approach for locating patterns and has different preprocessing and matching requirements.

One of the challenges was maintaining separate implementations without creating conflicts between the algorithms. The algorithms needed to work with the recipe data while keeping the overall search process understandable and manageable. For example, Naive Pattern Matching performs direct character comparisons, while KMP requires an LPS array for preprocessing. Rabin-Karp uses hashing to compare patterns, whereas Aho-Corasick uses a Trie-based structure with failure links.

Another challenge was ensuring that the pattern-matching implementations could be tested independently. Different test inputs were required to verify whether each method was producing the expected result. The implementation therefore required careful organization of the search-related methods and appropriate test cases.

The algorithms were maintained as separate components within the core `RecipeFinder.java` implementation. This allowed the project to demonstrate multiple pattern-matching techniques while keeping the graphical interface independent of the individual algorithm implementations.

6.1.2 Handling a Large Dataset

The recipe dataset was increased to **100,000 records**, which introduced additional considerations compared with a smaller dataset. Loading a larger number of records requires more initialization time and memory than loading a small collection.

A major challenge was avoiding unnecessary repeated processing of the dataset. If the recipe file were repeatedly read whenever the user entered a query, the search interface would become inefficient, particularly when the user performs multiple searches.

The application therefore loads the recipe records during initialization and stores the parsed

recipes in memory. The required search structures are then prepared from the loaded data. This approach avoids repeatedly reopening and parsing the complete recipe file for every search operation.

The larger dataset also required attention to the autocomplete mechanism. Constructing the Trie repeatedly for every search would introduce unnecessary overhead. Therefore, the Trie is constructed once and reused for subsequent prefix searches.

Testing with the larger dataset also required checking whether the existing search and recommendation features continued to function correctly after the increase in the number of records.

6.1.3 Autocomplete

Implementing autocomplete was another important challenge because suggestions must be generated while the user is entering a query. A direct scan of all recipe names for every character entered would result in repeated processing of the dataset.

The project uses a Trie data structure for prefix-based autocomplete. However, the Trie must first be constructed using the available recipe names. The challenge was to ensure that this structure was built at the appropriate stage and then reused instead of being reconstructed during every user interaction.

The Java Swing interface also needs to detect changes in the search field. As the user enters characters, the GUI requests suggestions corresponding to the current prefix. This requires coordination between the Swing input component and the Trie-based search functionality.

Another consideration was the number of suggestions displayed to the user. A common prefix may correspond to a large number of recipes, particularly in a dataset containing 100,000 records. Therefore, displaying an unrestricted number of suggestions would make the interface difficult to use. The application limits the displayed suggestions while retaining the Trie structure for efficient prefix lookup.

6.1.4 Dataset Parsing

The recipe dataset contains multiple fields for each recipe rather than storing every recipe as a single line. A recipe record contains information such as the recipe name, ingredients, instructions, preparation time and cooking time.

Therefore, the application needs to identify where one recipe begins and where the next recipe begins. It must also associate each field with the correct recipe. Incorrect parsing could result in ingredients being associated with the wrong recipe or incomplete recipe objects.

The parser was designed to process the structured format consistently. The Recipe: field

identifies the recipe name, while the corresponding ingredient, instruction and time fields are collected as part of the same record.

The large number of records made consistency particularly important. A small parsing problem could potentially affect many records. Therefore, the dataset structure had to remain consistent while extending the collection to 100,000 records.

6.1.5 Fuzzy Search

Exact string matching is not always sufficient for a recipe-search application. Users may enter an incomplete or slightly different spelling of a recipe name. To support such cases, the project implements fuzzy searching using Levenshtein Edit Distance.

The challenge with fuzzy search is that edit distance requires character-level comparison between strings. When there are many candidate recipe names, performing these calculations can require significant processing.

The system calculates the number of insertions, deletions and substitutions required to transform one string into another. The resulting distance is then converted into a similarity value so that the application can determine how closely the query matches a recipe name.

The implementation also needs to handle strings of different lengths and avoid incorrect calculations when one of the input strings is empty. Appropriate validation and string-length handling are therefore required.

Since fuzzy matching may involve multiple independent similarity calculations, the project also includes parallel similarity processing using Java's `ExecutorService` and `Future` mechanisms. This introduced an additional implementation consideration because the parallel tasks and their results need to be managed correctly.

6.1.6 Ingredient Matching

Ingredient-based searching presented a different type of problem from recipe-name searching. A normal recipe-name query is generally a string-matching problem, whereas an ingredient query may contain several ingredients that need to be compared with the ingredients available in recipes.

For example, a user may provide ingredients such as:

chicken, onion, tomato

The system needs to determine how these requested ingredients relate to the ingredients of available recipes. Differences in capitalization and textual representation also need to be considered during matching.

The project addresses this problem using a matching approach based on bipartite matching. The requested ingredients can be treated as one set and the recipe ingredients as another set, with

possible matching relationships between them.

The challenge was therefore not only implementing the matching algorithm but also converting the textual ingredient information into a form suitable for matching. The result must then be presented in a meaningful way to the user, such as reporting the number of successfully matched ingredients.

6.1.7 Greedy Coverage

Ingredient coverage is different from simply finding one recipe containing a particular ingredient. The objective is to identify recipes that collectively cover a requested set of ingredients.

This creates a set-cover-style problem. The challenge is that selecting recipes in an arbitrary order may not provide good coverage. The project therefore uses a Greedy Set Cover approach.

At each stage, the system considers the ingredients that are still uncovered and evaluates candidate recipes according to the number of currently uncovered ingredients they can cover. A suitable recipe is selected, its covered ingredients are removed from the remaining set, and the process continues.

Implementing this operation required maintaining the set of uncovered ingredients and updating it after every selection. The application also needs to terminate appropriately when all requested ingredients are covered or when no candidate recipe can provide additional coverage.

Since Greedy Set Cover is an approximation approach, the selected recipes depend on the greedy selection process rather than guaranteeing an optimal solution for every possible input.

6.1.8 GUI Integration

Another major challenge was integrating the existing DSA implementation with the Java Swing graphical interface. The project already contained the core search and recommendation algorithms, so the GUI needed to use these existing methods rather than duplicate their functionality.

The graphical interface contains components such as text fields, buttons, suggestion areas and result panels. These components must communicate with the underlying `RecipeFinder.java` functionality.

The autocomplete feature required the GUI to respond to changes in the search field, while search and recommendation operations required user actions to trigger the appropriate DSA methods.

Another consideration was keeping the interface simple. The user should not need to understand which algorithm is being executed internally. Instead, the GUI provides user-oriented operations such as recipe search, ingredient search, greedy coverage and random recommendation.

This separation between the user interface and the core DSA implementation helped maintain the organization of the project. The Swing interface acts as the interaction layer, while RecipeFinder.java remains responsible for the core data processing and algorithmic operations.

6.2 Solutions Applied

During the development of the Recipe Finder system, several implementation challenges were encountered while integrating multiple Data Structures and Algorithms with a 100,000-record recipe dataset and Java Swing interface. The following solutions were applied to address these challenges.

6.2.1 Solution for Multiple Pattern-Matching Algorithms

The project required the integration of four pattern-matching techniques: Naive Pattern Matching, Knuth-Morris-Pratt (KMP), Rabin-Karp and Aho-Corasick. Since each algorithm follows a different matching strategy, the implementations were organized as separate methods within the core DSA module.

Naive Pattern Matching was implemented as a straightforward character-by-character comparison method. KMP was implemented using the Longest Prefix Suffix (LPS) array to avoid unnecessary comparisons. Rabin-Karp was implemented using hashing and a rolling hash mechanism. Aho-Corasick was implemented using a Trie-like structure and failure links.

The algorithms are used internally by the application rather than being exposed to the user through an algorithm-selection menu. This keeps the graphical interface simple while allowing the project to demonstrate multiple pattern-matching techniques.

6.2.2 Solution for Handling the 100,000-Record Dataset

The dataset was expanded to 100,000 recipe records to provide a larger environment for testing the implemented algorithms.

To avoid repeatedly reading the dataset, the application loads the recipe file during initialization and stores the parsed recipe objects in memory. The recipe names are then used to construct the autocomplete Trie.

The Trie is built once after loading the dataset instead of being rebuilt every time the user enters a character. This reduces unnecessary repeated processing during autocomplete operations.

The dataset is stored in:

data/recipes.txt

and follows a structured record format containing recipe name, ingredients, instructions, preparation time and cooking time.

6.2.3 Solution for Autocomplete

Autocomplete was implemented using a Trie data structure.

Instead of scanning all 100,000 recipe names whenever the user types a character, the application traverses the Trie according to the entered prefix. Once the prefix node is reached, the system collects the available recipe names below that node.

The Java Swing `DocumentListener` is used to detect changes in the search field. Whenever the input changes, the GUI requests suggestions from the already constructed Trie.

For example, when the user enters:

bir

the system traverses:

$b \rightarrow i \rightarrow r$

and retrieves matching recipe-name suggestions.

This approach provides an efficient prefix-search mechanism and makes autocomplete suitable for interactive use.

6.2.4 Solution for Dataset Parsing

The recipe dataset contains multiple fields for every recipe. Therefore, a structured parsing approach was used.

The parser identifies the Recipe: header as the beginning of a new recipe record. The corresponding ingredient, instruction, preparation-time and cooking-time fields are then associated with that recipe.

The parsed information is stored in Recipe objects so that the rest of the application can work with structured data instead of repeatedly processing raw text.

This separation between **file parsing** and **algorithm processing** also improves the organization of the project.

6.2.5 Solution for Fuzzy Search

Exact matching cannot identify recipes when the user enters a slightly different spelling or incomplete query. To address this issue, Levenshtein Edit Distance was implemented.

The algorithm calculates the minimum number of insertions, deletions and substitutions required to transform one string into another.

The resulting distance is converted into a similarity percentage using:

Similarity =

$$(1 - \text{Edit Distance} / \text{Maximum String Length}) \times 100$$

The application uses a similarity threshold for accepting fuzzy results. The current GUI uses a **70% threshold**.

This allows the system to provide a useful result even when the user's query is not an exact recipe-name match.

6.2.6 Solution for Ingredient Matching

Ingredient-based searching is different from normal recipe-name searching because a query may contain multiple ingredients.

The requested ingredients are first normalized so that differences in capitalization and unnecessary characters do not prevent matching.

The ingredients are then treated as two sides of a matching problem:

Requested Ingredients



Matching Relationships



Recipe Ingredients

Bipartite matching is used to determine compatible ingredient matches.

This provides a structured way of comparing multiple requested ingredients against the ingredients available in a recipe.

6.2.7 Solution for Ingredient Coverage

Ingredient matching and ingredient coverage are handled as separate operations.

For coverage, the requested ingredients are initially considered uncovered. The Greedy Set Cover approach repeatedly examines candidate recipes and determines how many currently uncovered ingredients each recipe can cover.

The recipe providing the greatest useful coverage is selected. The ingredients covered by that recipe are removed from the uncovered set, and the process continues.

The procedure can be summarized as:

1. Store requested ingredients.

2. Mark all requested ingredients as uncovered.
3. Examine available recipes.
4. Calculate coverage for each recipe.
5. Select the recipe with maximum coverage.
6. Remove covered ingredients.
7. Repeat until coverage is complete
or no useful recipe remains.

This approach provides an approximation for the ingredient-coverage problem.

6.2.8 Solution for Random Recommendation

A separate randomized recommendation function was implemented for users who want a recipe without entering a specific search query.

The application selects a random index from the loaded recipe collection and retrieves the corresponding recipe.

Because the recipes are already loaded into memory, selecting a recipe by index does not require reading the dataset file again.

The process is conceptually:

Generate random index



Access recipe at index



Display recipe

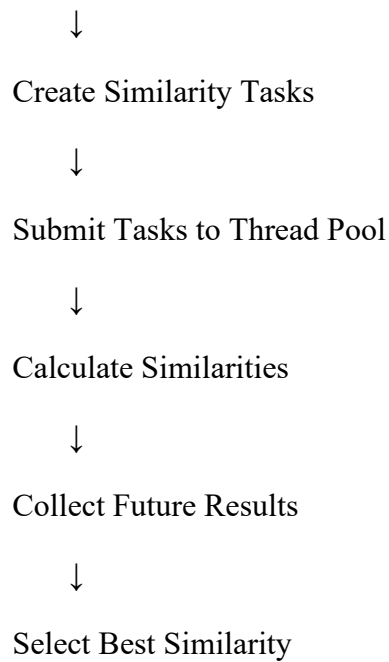
This keeps the random recommendation operation simple and independent from the search algorithms.

6.2.9 Solution for Parallel Similarity Processing

Similarity calculations can be performed independently for different recipes. The project therefore uses Java's ExecutorService and Future mechanisms for parallel similarity processing.

The workflow is:

Recipe Collection



This allows independent similarity calculations to be processed concurrently.

However, parallel execution also introduces thread-management overhead. Therefore, the implementation is treated as a demonstration of parallel processing rather than assuming that parallel execution will always be faster for every dataset size.

6.2.10 Solution for Java Swing Integration

The existing DSA implementation was kept separate from the graphical interface.

The core DSA functionality is implemented in:

`src/RecipeFinder.java`

while the GUI is implemented in:

`src/RecipeFinderGUI.java`

The Swing interface provides controls for:

- recipe search;
- autocomplete;
- ingredient search;
- greedy coverage;
- random recommendation;
- displaying recipe information.

This separation allows the algorithms to remain organized while providing users with a simple interface.

6.2.11 Solution for Invalid and Empty Input

Input validation was included to handle cases where the user submits an empty query, invalid recipe name or invalid ingredient input.

For example, when no recipe name is entered, the system can display a validation message rather than attempting unnecessary search operations.

Similarly, if a recipe cannot be found, the GUI provides an appropriate result instead of terminating the application.

This improves the reliability and usability of the system.

6.2.12 Overall Effect of the Solutions

The solutions applied during development allowed the different components of Recipe Finder to operate together as a single application.

The dataset is loaded and parsed first, after which the Trie and other required structures are prepared. The Java Swing interface then provides access to the search and recommendation operations.

The resulting system combines:

- efficient prefix searching through Trie;
- multiple pattern-matching algorithms;
- fuzzy matching through Levenshtein distance;
- graph-based ingredient matching;
- greedy ingredient coverage;
- randomized recommendation;
- parallel similarity processing.

These solutions helped transform the individual DSA implementations into an integrated recipe-search application capable of working with the current 100,000-record dataset.

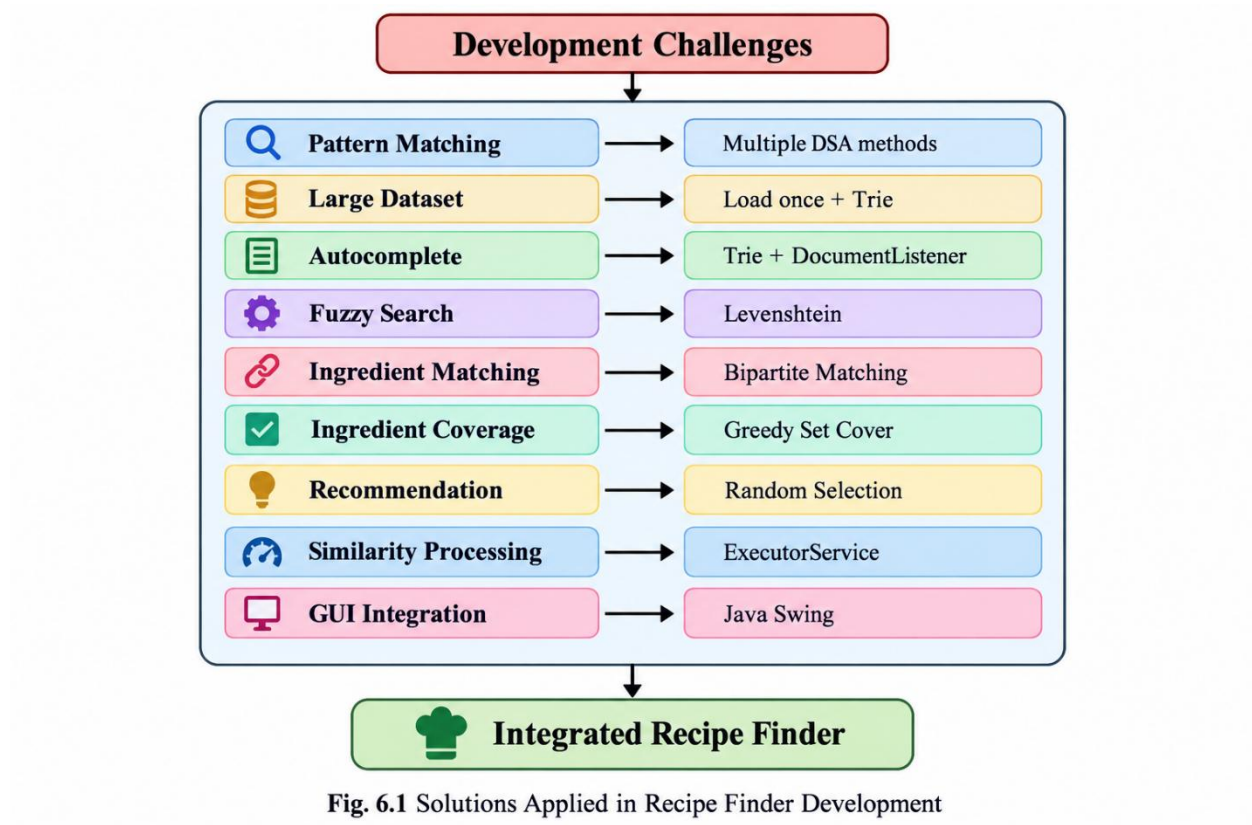


Fig. 6.1 Solutions Applied in Recipe Finder Development

6.3 Limitations

Although the Recipe Finder system successfully integrates multiple Data Structures and Algorithms and supports a 100,000-record dataset, the current implementation has certain limitations. These limitations are related to data storage, algorithm implementation, processing overhead, application platform, and dataset provenance.

6.3.1 Text-File-Based Data Storage

The application currently uses a text-file dataset stored in data/recipes.txt rather than a database management system. The recipe records are loaded and parsed during application initialization. Although this approach is suitable for the current project and simplifies dataset management, it does not provide database features such as indexed queries, transactions, concurrent updates, or efficient persistent record management.

For a substantially larger dataset, a database or other persistent storage mechanism could provide

more efficient data management and querying.

6.3.2 Aho-Corasick Implementation

The current Aho-Corasick implementation has a specific limitation that should be stated accurately. The automaton is constructed from the supplied pattern during the verification process. Therefore, the current implementation should not be described as maintaining a permanent multi-pattern Aho-Corasick index over the complete recipe dataset.

A future implementation could construct and maintain a reusable multi-pattern automaton if the application requires repeated matching against a fixed collection of patterns.

6.3.3 Parallel Fuzzy-Search Overhead

The parallel fuzzy-search implementation creates similarity-processing tasks for the recipe candidates. Although independent similarity calculations can be processed concurrently, creating and managing multiple tasks introduces additional overhead.

For smaller workloads, this task-management overhead may reduce or offset the benefit of parallel execution. Therefore, parallel processing does not automatically guarantee faster execution for every query or dataset size.

6.3.4 Greedy Set Cover Approximation

The ingredient-coverage functionality uses a **Greedy Set Cover** strategy. Greedy Set Cover is an approximation approach that repeatedly selects a recipe providing useful coverage of currently uncovered ingredients.

Because it is a greedy approximation, the method does **not guarantee the globally minimum number of selected recipes** in every possible case. Its purpose in the current project is to demonstrate the application of greedy optimization to the ingredient-coverage problem.

6.3.5 Desktop Application Platform

The current Recipe Finder application uses **Java Swing** and is therefore implemented as a desktop application. It is not currently a web application or mobile application.

Users need a compatible Java environment to execute the application. A future version could provide a web-based or mobile interface to make the application accessible through additional platforms.

6.3.6 Dataset Provenance

The current project documentation identifies the recipe dataset used by the application as data/recipes.txt. If externally sourced real-world recipe data is required, the source and provenance of that dataset should be documented separately.

Therefore, the project should not make unsupported claims about external dataset sources. Future versions can improve reproducibility by documenting the original source, collection method, licensing information, preprocessing steps, and transformation process associated with the recipe data.

6.3.7 Summary of Limitations

The limitations of the current system do not prevent it from demonstrating the intended DSA concepts, but they identify areas where the application can be improved. The current implementation prioritizes demonstrating algorithmic techniques, integrating them into a single Java application, and processing the available 100,000-record dataset. Future development can address these limitations through improved storage, reusable pattern indexes, optimized parallel processing, advanced optimization techniques, and additional application platforms.

CHAPTER 7 — CONCLUSION & FUTURE WORK

7.1 Conclusion

The **Recipe Finder** project demonstrates the practical application of Data Structures and Algorithms to the problem of searching, matching, comparing, and recommending recipes from a large recipe dataset. The system was developed as a Java-based application and currently operates on a **100,000-record recipe dataset** stored in data/recipes.txt.

The project combines several algorithmic techniques rather than depending on a single search method. Different algorithms are used for different types of operations. Naive Pattern Matching, Knuth-Morris-Pratt (KMP), Rabin-Karp, and Aho-Corasick are used for pattern-matching operations. A Trie is used to support prefix-based autocomplete, allowing recipe suggestions to be generated from partial user input.

For approximate searching, the project uses **Levenshtein Edit Distance**. This allows the system to calculate the similarity between a user's query and recipe names. As a result, the application can support fuzzy searching in addition to exact matching.

The project also demonstrates the use of algorithms beyond conventional string searching. **Bipartite Matching** is used for ingredient-based matching, where requested ingredients are compared with recipe ingredients. **Greedy Set Cover** is used for ingredient coverage, where recipes are selected according to their ability to cover currently uncovered requested ingredients.

A randomized recommendation feature is also included. This allows the system to select a recipe from the loaded collection without requiring a specific recipe-name query. In addition, parallel similarity processing is implemented using Java concurrency mechanisms for independent similarity calculations.

The system provides several user-oriented operations, including:

- exact recipe searching;
- case-insensitive searching;
- Trie-based autocomplete;
- fuzzy searching;
- ingredient matching;
- greedy ingredient coverage;
- random recipe recommendation.

The project also provides a **Java Swing graphical user interface**. The GUI provides an accessible interaction layer for the implemented DSA functionality. Instead of requiring users to interact directly with individual algorithms, the interface presents operations such as recipe search, autocomplete, ingredient search, greedy coverage, and random recommendation.

The use of a 100,000-record dataset provides a larger testing environment for the implemented algorithms. The project therefore demonstrates not only the implementation of individual DSA techniques but also their integration into a single application. Loading and parsing the dataset,

constructing reusable search structures, processing user input, performing algorithmic operations, and displaying results are combined into one workflow.

The project also demonstrates the relationship between theoretical concepts and practical implementation. For example, the Trie concept is applied to autocomplete, dynamic programming is applied to edit-distance calculation, graph-based matching is applied to ingredient matching, and greedy optimization is applied to ingredient coverage.

Overall, the Recipe Finder project provided practical experience in designing, implementing, testing, and integrating multiple Data Structures and Algorithms in Java. It also provided experience in handling a large structured dataset, developing a graphical interface, performing algorithmic testing, managing project files using Git/GitHub, and documenting the development process.

The completed system therefore serves as an integrated demonstration of how different algorithmic approaches can be selected and combined according to the requirements of a real-world-style search and recommendation application.

7.2 FUTURE WORK

The current Recipe Finder system provides the required searching, matching, recommendation, and GUI functionality. Several additional features can be considered for future development.

7.2.1 Improved Search Ranking

Future versions can improve the ordering of search results by considering multiple factors such as exact-match priority, prefix similarity, fuzzy similarity, and ingredient relevance. This could help organize a larger number of matching recipes according to their relevance to the user's query.

7.2.2 Personalized Recipe Recommendations

The recommendation component can be extended to consider user preferences and previous selections. Future versions could use information such as preferred ingredients, frequently selected recipes, or dietary requirements to generate more personalized recommendations.

7.2.3 Nutritional and Dietary Information

Future versions can include nutritional information such as calories, protein, carbohydrates, fats, and other nutritional attributes when reliable data is available. Dietary preferences such as vegetarian, vegan, gluten-free, or low-calorie requirements could also be incorporated into recipe filtering.

7.2.4 Larger Real-World Datasets

The system can be extended to support larger and more diverse recipe collections. Future datasets should have clearly documented provenance and consistent metadata. This would provide additional opportunities for evaluating algorithm performance under different data distributions and dataset sizes.

7.2.5 Graph-Based Recipe and Ingredient Relationships

A graph representation could be introduced to represent relationships between recipes and ingredients. Recipes could be connected to their ingredients, while common ingredients could establish relationships between recipes. Such a structure could support additional recommendation and ingredient-exploration operations.

7.2.6 Web-Based or Mobile Interface

The current application uses Java Swing as its graphical interface. A future version could provide a web-based or mobile interface so that the recipe-search functionality can be accessed from different devices. The existing DSA logic could be separated further from the presentation layer to support such interfaces.

7.2.7 Automated Performance Benchmarking

Future development can include automated benchmarking of the implemented algorithms. Search time, preprocessing time, memory usage, and similarity-processing time could be measured for different dataset sizes. This would allow the practical performance of the algorithms to be studied more systematically.

7.2.8 Larger Automated Test Suites

The current functionality can be extended with additional automated test cases. Future test suites could include more recipe names, different query lengths, spelling variations, empty inputs, invalid inputs, ingredient combinations, and larger numbers of candidate recipes.

7.2.9 Optimization of Parallel Similarity Processing

The parallel similarity component can be further optimized by studying thread-pool size, task creation overhead, result collection, and workload distribution. Different approaches could be compared to determine how parallel processing behaves for different dataset sizes and query workloads.

7.3 SKILLS LEARNED

The development of Recipe Finder provided practical experience across algorithm design, Java programming, data processing, graphical interface development, testing, and technical documentation.

7.3.1 Problem Analysis

The project provided experience in converting a real-world-style recipe-search problem into smaller computational problems. Searching, autocomplete, fuzzy matching, ingredient matching, coverage, and recommendation were identified as separate requirements that could be addressed using different algorithmic approaches.

7.3.2 Algorithm Selection

The project provided practical understanding of selecting algorithms according to the nature of a problem. Different pattern-matching algorithms were studied and implemented, while Trie,

Levenshtein Distance, Bipartite Matching, Greedy Set Cover, randomization, and parallel processing were applied to their respective tasks.

7.3.3 String Matching

Implementation of Naive Pattern Matching, KMP, Rabin-Karp, and Aho-Corasick provided practical experience with string-processing techniques and their different matching strategies.

7.3.4 Trie Implementation

Working with the Trie data structure provided practical understanding of prefix-based storage and searching. The Trie was applied to recipe-name autocomplete so that suggestions could be generated from partial input.

7.3.5 Dynamic Programming

The implementation of Levenshtein Edit Distance provided experience with dynamic programming. The project demonstrated how a problem can be divided into smaller subproblems and represented using a dynamic-programming table.

7.3.6 Edit-Distance Calculation

The project provided practical experience in calculating the number of insertions, deletions, and substitutions required to transform one string into another. This concept was then used to support fuzzy recipe searching.

7.3.7 Bipartite Matching

The ingredient-matching component provided experience with representing matching relationships between two sets. The project demonstrated how bipartite matching can be applied to an ingredient-recipe matching problem.

7.3.8 Greedy Optimization

The Greedy Set Cover implementation provided experience with iterative decision-making. At each stage, the algorithm selects a recipe that provides useful coverage of the currently uncovered ingredients.

7.3.9 Randomized Algorithms

The random recommendation feature provided practical experience with randomized selection. A recipe can be selected from the loaded collection without requiring a specific search query.

7.3.10 Java Concurrency

Parallel similarity processing provided experience with Java concurrency concepts, including `ExecutorService` and `Future`. The project demonstrated how independent computational tasks can be submitted and their results collected.

7.3.11 Java Swing

Developing the graphical interface provided experience with Java Swing components and event-driven interaction. Search fields, buttons, suggestion areas, and result displays were connected to the underlying DSA functionality.

7.3.12 Dataset Parsing

The project provided experience in reading and parsing structured text data. Recipe records containing multiple fields had to be identified and converted into usable recipe objects.

7.3.13 Testing

Testing the project provided experience in verifying different functional requirements. Exact searches, case variations, autocomplete, fuzzy queries, ingredient matching, greedy coverage, random recommendation, empty input, invalid input, and exit operations were considered during testing.

7.3.14 Git and GitHub

The project also provided experience in using Git and GitHub for source-code management. Version control helped organize changes to Java source files, dataset files, documentation, test results, and project resources.

7.3.15 Technical Documentation

Preparing the project documentation provided experience in presenting problem statements, requirements, system design, algorithms, implementation details, test results, limitations, conclusions, and future work in a structured technical format.

REFERENCES

- [1] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977. DOI: 10.1137/0206024.
- [2] R. M. Karp and M. O. Rabin, “Efficient Randomized Pattern-Matching Algorithms,” *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987. DOI: 10.1147/rd.312.0249.
- [3] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975. DOI: 10.1145/360825.360855.
- [4] E. Fredkin, “Trie Memory,” *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960. DOI: 10.1145/367390.367400.
- [5] V. I. Levenshtein, “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” *Soviet Physics Doklady*, vol. 10, pp. 707–710, 1966.
- [6] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973. DOI: 10.1137/0202019.
- [7] Oracle Java Documentation, Java Swing and standard Java libraries used for Java GUI and application development.
- [8] Recipe Finder project source code and test evidence: src/RecipeFinder.java, src/RecipeFinderGUI.java, data/recipes.txt, and project results/ directory.

